A Hybrid Approach to Integrating Deterministic and Non-deterministic Concurrency Control in Database Systems

ABSTRACT

Deterministic and non-deterministic concurrency control algorithms have shown respective advantages under diverse workloads. Thus, a natural idea is to blend them together. However, because deterministic algorithms work with stringent assumptions, e.g., batched execution and non-interactive transactions, they hardly work together with non-deterministic algorithms. To address this issue, we propose HDCC, a hybrid approach that adaptively employs Calvin, a classic deterministic algorithm, and Silo, a classic non-deterministic algorithm, in the same database system. Calvin and Silo take completely different concurrency control schemes and logging schemes. Consequently, we propose the lock-sharing and global validation mechanisms to ensure the serializable schedule of transactions, and propose a two-log-interleaving mechanism to ensure correct recovery upon failures. Further, we introduce a rule-based assignment mechanism in which a transaction is adaptively scheduled by either Calvin or Silo, tailoring its optimal choice to different workload scenarios. Extensive experiments conducted over both TPC-C and YCSB benchmarks show that HDCC outperforms the state-of-the-art hybrid approaches by a factor of up to 3.1×.

PVLDB Reference Format:

. A Hybrid Approach to Integrating Deterministic and Non-deterministic Concurrency Control in Database Systems. PVLDB, 14(1): XXX-XXX, 2020.

doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/dbiir/HDCC.

1 INTRODUCTION

Concurrency control algorithms play a pivotal role in ensuring consistency and isolation in database systems. These algorithms fall into two categories: deterministic and non-deterministic. Deterministic algorithms process transactions in batches, where transactions within each batch are ordered, often based on criteria such as arrival time. A transaction, denoted as T, is scheduled for execution only if all conflict transactions that are ordered prior to T have been completed. Notably, if multiple such transactions exist, they can be executed in parallel, thereby improving concurrency. In contrast, non-deterministic algorithms schedule each transaction individually. The order of concurrent transactions in an equivalent serializable schedule [6] is predetermined at the beginning of a transaction or dynamically determined during execution.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

There is no single concurrency control algorithm that is universally optimal for all workloads [7, 20, 49]. Deterministic algorithms [14, 15, 42] excel in processing either high contention workloads or distributed transactions or both, due to their deterministic schedule and without a need for a two-phase commit protocol (2PC). However, deterministic algorithms work under stringent assumptions, like batched execution and predeclared read/write sets, limiting their applicability in reality. Although a few works explore eliminating some of the assumptions, extra overheads are introduced, making them be sub-optimal. For example, Calvin [42] introduces a reconnaissance mechanism by pre-executing the batched transactions, but its performance suffers, especially in the workload with long-running transactions [36]. Aria [30] proposes a deterministic optimistic concurrency control, but at a cost of introducing a 2PC-like protocol as well as a higher abort rate under high contention workload [25, 46]. Worse still, deterministic algorithms cannot support interactive transactions. On the contrary, non-deterministic algorithms such as optimistic concurrency control (OCC) and its variants ([28, 43]), two-phase locking (2PL) and its variants ([8, 19]) work without the aforementioned assumptions. They excel in low or medium contention workloads. However, their performance suffers when processing high contention workloads or distributed transactions [20, 49].

To maximize the respective advantages, we are motivated to develop a hybrid approach that integrates deterministic and non-deterministic algorithms into the same database system. Thus far, although a wide array of hybrid approaches exist, they primarily focus on a combination of non-deterministic algorithms only. Many notable works aim to merge multiple concurrency control algorithms into a singular, innovative algorithm. Examples include MVOCC [12, 27], MV2PL [47] and MVTO [34, 35] that integrate multi-version concurrency control (MVCC) with OCC, 2PL, and TO, respectively. In contrast, other studies [37–40, 44, 48] seek to incorporate multiple concurrency control algorithms within a single database system, with each operating independently and capitalizing on its particular strengths. These algorithms work either at the data item or transaction granularity level. At the data item granularity [40, 41], certain data items, such as frequently accessed (hot) items, are scheduled by one algorithm, while others are handled by different algorithms. At the transaction granularity [37–39, 44], each transaction is scheduled exclusively by a single algorithm. The key design challenge is to ensure serializability among conflict transactions that are scheduled by different concurrency control algorithms. To the best of our knowledge, Snapper [29] is the only work that mixes deterministic and non-deterministic algorithms. In Snapper, transactions with pre-declared read/write sets are scheduled and executed in batches by Calvin, while other transactions are individually scheduled by 2PL. Snapper introduces an extra validation phase for each transaction T scheduled by 2PL, to check whether T can commit or not by comparing it with a batch of

transactions scheduled by Calvin. Snapper is sub-optimal because, its concurrency control algorithms do not fully leverage their individual superiority (e.g., Calvin should be used for high contention workloads), and it leads to a higher abort rate due to the coarse-grained validation that takes a batch of transactions as a whole.

In this paper, we propose HDCC, a hybrid approach that incorporates Calvin and OCC into the same database system. We choose Calvin as the deterministic algorithm because of its popularity and adoption in real database systems, like FaunaDB [2, 42]. The reason why we choose OCC as the non-deterministic algorithm is three-fold. First, it is widely used in many real database systems, like Azure Cosmos DB [1, 3, 5]. Second, it excels in low and medium contention workloads [49] and complements Calvin perfectly. Third, if a transaction scheduled by OCC aborts, it can be rescheduled by Calvin due to its known read/write set. In HDCC, each transaction is exclusively scheduled by either Calvin or OCC during its entire execution. The assignment of Calvin and OCC is rule-based, tailoring its optimal choice to different workloads. For example, Calvin is used to process distributed transactions with pre-declared read/write sets, as well as local transactions with pre-declared read/write sets over hot data items.

Because Calvin and OCC adopt completely different concurrency control and logging schemes, integrating them within the same system is not straightforward. The key challenge is to ensure both serializability and correct failure recovery. **(1) Serializability.** Although transactions scheduled by Calvin or OCC individually are serializable, conflict transactions, with some scheduled by Calvin and others by OCC, may not be serializable. For example, consider two transactions T_1: $W_1(x)W_1(y)$ and T_2: $R_2(x)R_2(y)$, which are scheduled by Calvin and OCC, respectively. Suppose x and y are located in node 1 and node 2, respectively. As a result, T_1 and T_2 are decomposed into two sub-transactions each. Since T_1 and T_2 conflict in both node 1 and node 2, it is possible that on node 1, T_1 is ordered before T_2, but on node 2, T_1 is ordered after T_2. Clearly, such a schedule is not serializable. To solve this problem, we propose two mechanisms, namely lock-sharing and global validation, which are integrated into OCC. Given a transaction T scheduled by OCC, the lock-sharing mechanism in each node helps check which conflict transactions scheduled by Calvin are ordered before T. If T is a distributed transaction, the global validation mechanism further verifies whether T and its conflicting transactions scheduled by Calvin form a cycle across nodes. If a cycle exists, T aborts; otherwise, T is able to commit. We theoretically prove that transactions scheduled by HDCC are serializable. Compared to Snapper, HDCC achieves a finer-grained schedule, which leads to a lower abort rate. **(2) Recovery.** Logging is the key technology to ensure failure recovery. However, simply applying the logging mechanisms of Calvin and OCC separately cannot correctly restore the database upon failure. Note that Calvin processes transactions in batches and writes all the logs of batched transactions *before* scheduling them to execute. In contrast, OCC processes each transaction individually and writes the logs *after* scheduling it to execute but before scheduling it to commit. Consequently, the serializable order among transactions scheduled by Calvin and OCC, respectively, cannot be captured based on the logs. When a failure occurs, the committed OCC and Calvin transactions may not be correctly recovered by replaying the redo logs. Intuitively, a baseline approach is to use an additional log file to maintain and record the order once a transaction commits. However, this approach could incur extra maintenance and logging overhead. To address this problem, we design a two-log-interleaving mechanism. In this mechanism, upon the commit of an OCC transaction, we explicitly maintain and log the committed Calvin transactions that are ordered before it. By doing this, we not only capture the order between Calvin and OCC transactions but also record this order in a lightweight manner. Upon a failure, we follow the order recorded in the OCC log and interleave the replay of OCC and Calvin logs to recover the committed Calvin and OCC transactions. We provide the theoretical proof along with the TLA+ specification [33] in the appendix to demonstrate the correctness of our failure recovery mechanism.

To summarize, we make the following contributions.

- We propose HDCC, a hybrid concurrent control algorithm that adaptively employs OCC and Calvin. HDCC is equipped with two mechanisms, lock-sharing and global validation, to ensure the serializable schedule of transactions.
- We design a two-log-interleaving mechanism that helps correctly recover the committed Calvin and OCC transactions once a failure occurs. We provide the theoretical proof along with the TLA+ specification to demonstrate its correctness.
- We propose a rule-based assignment mechanism, tailoring its optimal choice to different workloads. Two optimizations are further carefully designed to boost HDCC.
- We conduct extensive experiments over TPC-C and YCSB benchmarks. The results demonstrate that HDCC achieves significantly higher performance compared to the state-of-the-art hybrid approach, with a factor of up to 3.1× improvement.

2 PRELIMINARY

In this section, we review the concurrency control and logging schemes in Calvin and OCC which are the foundation of HDCC.

2.1 Calvin

In Calvin, each node is equipped with three components: the Sequencer, Scheduler, and Worker. The Sequencer collects transactions sent by clients, packs them into batches, determines the order of transactions, and writes batched transactions as the logical logs on disk in case of failures. The Scheduler requests locks for transactions one by one according to the order determined by the Sequencer. After acquiring all the locks, a transaction is scheduled for execution by a single Worker.

For illustration purposes, consider two transactions, T_1 and T_2, shown in Figure 1. Transaction T_1 writes data item x located in node N_1 and writes data item y located in node N_2. Transaction T_2 writes x. Initially, the Sequencer on N_1 (resp. N_2) receives T_1 (resp. T_2) and packs it into batch B (resp. $\hat{B}$). Every transaction in a Sequencer is allocated a globally unique, monotonically increasing transaction ID. Next, each Sequencer flushes the batched transactions (i.e., T_1 in N_1 and T_2 in N_2) as logical logs on disk. Subsequently, the Sequencer decomposes every transaction into multiple sub-transactions based on the nodes to be accessed and sends the sub-transactions to the Schedulers of the involved nodes. Note,

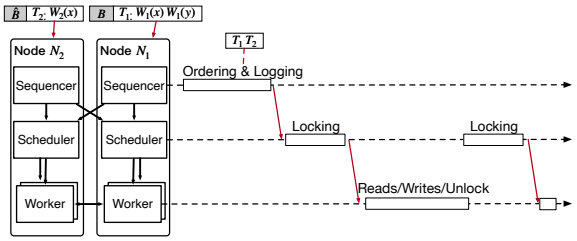


Figure 1: Distributed transaction processing using Calvin

the Sequencer would send a special message to a Scheduler if there does not exist sub-transactions to be sent. After one-round synchronization among Sequencers and Schedulers, each Scheduler is able to receive complete sub-transactions within the same batch. Next, the Scheduler requests locks for received sub-transactions in the ascending order of transaction IDs. For example, the Scheduler on N_1 requests locks for transactions in batches B and $\hat{B}$. If a transaction T has acquired all its locks, the Scheduler en-queues T to an execution queue Q. Meanwhile, the Worker de-queues a transaction T from Q one by one for execution. After reads/writes, the Worker releases the locks of T and sends a commit response to its initiated Sequencer.

Calvin utilizes a variant of ZigZag [10] to periodically create checkpoints. Data items are flushed to disk only during the checkpoint. Initially, Calvin selects a transaction ID *ckpID* as a pivot. Upon the time that all transactions with their transaction IDs smaller than or equal to *ckpID* have committed, Calvin flushes data items to disk, and the database reaches consistency. Note, during the checkpoint, Calvin allows transactions with their transaction IDs greater than *ckpID* to write data. To guarantee the consistency of the database, these writes are not directly applied to the database. Instead, they are maintained as separate copies. After the checkpoint completes, data items are overwritten by the copies. Once a node N fails, we need to recover all transactions recorded in the logical log of N (note these transactions are considered as committed). Due to the checkpoint, writes from transactions with their transaction IDs less than or equal to *ckpID* have been persisted. We only redo the transactions with their transaction IDs greater than *ckpID* by replaying the logical logs.

2.2 OCC

We adopt Silo [43], a variant of OCC and extend it to a distributed version. In Silo, a transaction T goes through three phases: read, validation, and write. In the read phase, Silo performs reads/writes without acquiring locks, and puts the data items that it reads/writes in the read/write set, respectively. In the validation phase, Silo acquires the locks for its write set and validates whether the data items in its read set have been modified. If no data items have been modified, it enters the write phase and applies the changes to the database. Note, read-only transactions skip the write phase.

The coordinator coordinates the execution of a distributed transaction T. T is decomposed into multiple sub-transactions, which are executed locally in the involved nodes using Silo. Consequently, T goes through three phases as well: 1) the execution phase corresponding to Silo's read phase, 2) the prepare phase corresponding to the validation phase, and 3) the commit phase corresponding to the write phase. Take transaction T_3 shown in Figure 2 for example.

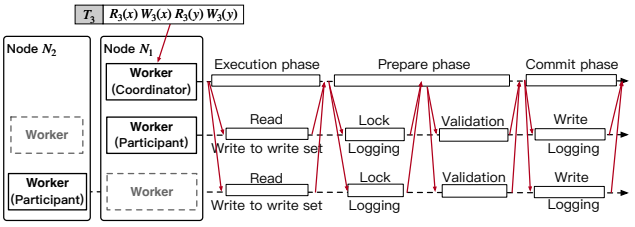


Figure 2: Distributed transaction processing using OCC

T_3 is decomposed into two sub-transactions, which are executed on N_1 and N_2, respectively. In the execution phase, the coordinator coordinates N_1 (often in terms of a Worker thread) as one participant in which T_1 reads and writes x without acquiring the lock, and puts x in the read set and write set. Besides, the coordinator coordinates N_2 as the other participant in which T_3 reads/writes y in the same way. After collecting all responses from the participants, the coordinator coordinates T_3 to enter the prepare phase. In the prepare phase, the coordinator notifies N_1 and N_2 to validate. To be specific, the coordinator first requires N_1 and N_2 to lock all the data items in the write set, i.e., x for N_1 and y for N_2. After acquiring the locks, N_1 and N_2 flush the logs to disk locally. Afterward, the coordinator sends the validation messages to N_1 and N_2 to check whether the data items in the read set, i.e., x for N_1 and y for N_2, have been modified. Because x and y have never been modified, the coordinator passes the validation. In the commit phase, the coordinator flushes the commit log and notifies N_1 and N_2 to do local commit. N_1 and N_2 write commit logs to disk, release the lock on x and y, and apply the changes to the database.

3 OVERVIEW

HDCC works in a shared-nothing system architecture. Each node is equipped with an in-memory database instance, and logically disaggregated into two layers: the storage layer and the compute layer. An overview of HDCC is given in Figure 3.

Storage layer. The storage layer in our design uses a hash partitioning method [20] to horizontally split data into partitions. Each partition is assigned exclusively to a single node within the cluster, and the storage layer of each node is accountable for managing access to its assigned partitions and writing logs on disk. To support concurrency control, we maintain additional metadata in memory, including lock tables, and statistics on the conflict of data items. This information aids in determining the most suitable concurrency control algorithms for a given transaction.

Compute layer. The compute layer within each node is responsible for selecting either OCC or Calvin to execute transactions, making its optimal choice adaptable to various workload scenarios. For this purpose, it employs four key components: (1) an Analyzer, (2) a Sequencer, (3) a Scheduler, and (4) a set of Workers.

- **Analyzer.** The Analyzer is responsible for receiving and analyzing transactions from clients or users. It analyzes the characteristics of these transactions and assigns the most appropriate algorithms. In our design, this assignment is rule-based, tailoring its optimal choice to different workloads. Transactions scheduled by OCC, referred to as OCC transactions, are directly routed by the Analyzer to Workers that implement OCC for concurrency control. Similarly, transactions scheduled by Calvin, known as Calvin transactions, are directly forwarded by the Analyzer to

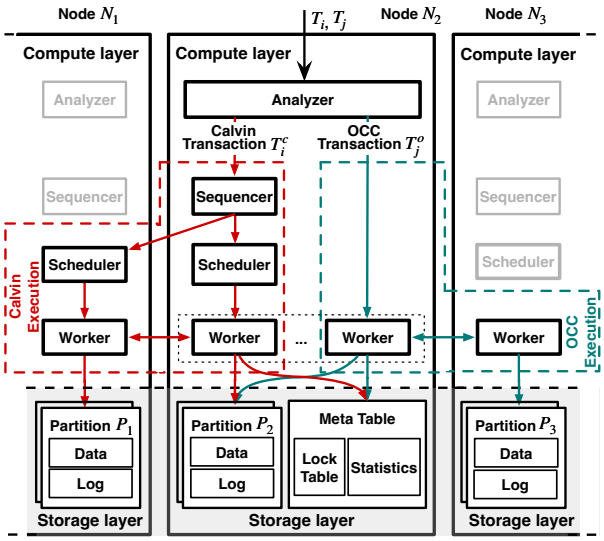


Figure 3: An overview of HDCC

Sequencer which employs Calvin for concurrency control. For brevity, a transaction T is represented as T^c (resp. T^o) if T is scheduled by Calvin (resp. OCC). Note, each transaction has a unique ID, which is a pair $\langle BID, TID\rangle$. TID is a monotonically increasing transaction ID assigned by the Analyzer. BID signifies the batch ID of the transaction. For Calvin transactions, BID will be assigned by the Sequencer later. For OCC transactions, BID is set to $NULL$. Given any two $T_i.ID$ and $T_j.ID$, we consider $T_i.ID > T_j.ID$, if $T_i.ID.TID > T_j.ID.TID$.

- **Sequencer.** The Sequencer is responsible for collecting Calvin transactions sent by Analyzers, determining their orders, batching them, and assigning BID into their IDs. Then, it writes the logs of these transactions following the same logic as described in Section 2.1. Subsequently, the Sequencer decomposes each transaction in the batch into sub-transactions (if any), and distributes them to the corresponding local or remote schedulers.
- **Scheduler.** The Scheduler is responsible for coordinating the execution of transactions, described in Section 2.1. After acquiring all necessary locks for a sub-transaction T, T is assigned to a single worker for execution. Notably, if there are multiple sub-transactions similar to T that have acquired their locks, they can be executed in parallel. Because conflict transactions in each Scheduler are executed one by one, and they follow the same ascending order of transaction IDs across Schedulers, the schedule of Calvin transactions is serializable.
- **Multiple Workers.** A Worker can act as an executor that executes sub-transactions or a coordinator that coordinates the execution of distributed OCC transactions. The Worker acts exclusively as a coordinator when it receives an OCC transaction from the Analyzer and determines that it is a distributed transaction. When a Worker acts as an executor and receives a Calvin sub-transaction from a Scheduler, it follows the same logic as described in Section 2.1 to perform local read and write operations, commits the sub-transaction, and subsequently releases the locks. If the Worker receives an OCC sub-transaction from a coordinator, it adopts the role of a participant in distributed transaction processing, processes local reads, writes, local validation, and commits, as described in Section 2.2. Conversely, when a Worker acts as a coordinator, it decomposes a transaction into multiple sub-transactions (if any), which are then disseminated to both local and remote Workers for processing. Once all the sub-transactions complete, the coordinator employs 2PC to commit the transaction. Both the coordinator and participants write the logs in this process following the same logic as described in Section 2.2. In our design, OCC does not flush dirty pages to disk after commit but relies on checkpoints to persist data as Calvin does. Note, we do not abort Calvin transactions. Instead, we abort OCC transactions as long as we cannot find an equivalent serializable schedule by committing them.

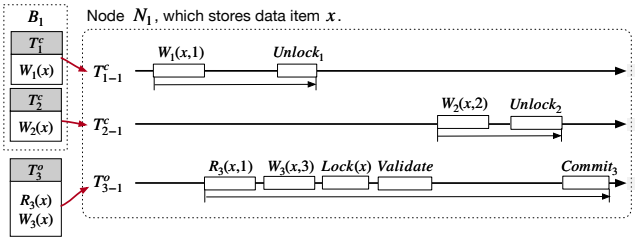


Figure 4: A running example of incorrect local schedule

4 HYBRID CONCURRENCY CONTROL

To start, we provide some necessary symbols used throughout the remainder of the paper. Given a transaction T_i^o (resp. T_i^c), let T_{i-k}^o (resp. T_{i-k}^c) be the sub-transaction located in node N_k. In the real execution, for any Calvin transaction T_i^c, all operations on data items located in N_k are first packed into sub-transaction T_{i-k}^c, which are then sent to N_k; for any OCC transaction T_i^o, operations on data items located in N_k could be progressively sent to N_k as sub-transaction T_{i-k}^o. Given two transactions T_i, T_j, if T_i first, and T_j next operate the same data item x, and one of the two operations is write, we say T_j depends on T_i. This dependency is denoted as $T_i \rightarrow T_j$. *For illustration purposes, when the context is clear, symbols T_{i-k}^c and T_i^c (resp. T_{i-k}^o and T_i^o) are used interchangeably.*

4.1 The lock-sharing mechanism

In a single node, there are multiple workers concurrently receiving and executing different sub-transactions. Due to the different execution mechanisms of Calvin and OCC, the conflict cannot directly be detected by each other, leading to inconsistent conflict orders between two sub-transactions.

Example 1. *Consider Figure 4. There are three sub-transactions in N_1. T_{1-1}^c: $W_1(x,1)$, T_{2-1}^c: $W_2(x,2)$, and T_3^o: $R_3(x)W_3(x)$. First, T_{1-1}^c writes 1 to x, followed by a read ($x = 1$) of T_{3-1}^o. Second, T_{3-1}^o writes x ($x = 3$) and adds x to its write set. Third, it enters the prepare phase where T_{3-1}^o acquires a lock on x, passes the validation since x has never been modified since its first read, and decides to commit. Fourth, sub-transaction T_{2-1}^c updates x to 2, and commits. This could happen because T_{2-1}^c cannot detect the conflict on x with T_{3-1}^o. Finally, T_{3-1}^o commits and applies its writes to the database. Consequently, the above schedule is not serializable because of the lost update of T_{2-1}^c.* □

Algorithm 1: OCC validation. The code highlighted in blue is related to global validation, described in Section 4.2.

```
1  function GET-LOCK-O():
2      foreach x in ws do
3          if !tryLock(x, EX) then return ⟨ABORT, NULL⟩;
4      end
5      return ⟨COMMIT, GET-LMAXCID()⟩;
6  end
7  function VALIDATE-LOCAL-O(gMaxCid):
8      foreach x in rs do
9          if isLocked(x, EX) or lock_table.x.wid! = rs.x.wid then
               return ABORT;
10     end
11     return VALIDATE-HYBRID(gMaxCid) ;
12 end
```

To address this issue, we introduce the lock-sharing mechanism to unify the metadata between Calvin and OCC so that conflicts among Calvin and OCC transactions can be detected. Specifically, we introduce two elements *lock* and *wid* for each data item in the lock table, operated by both Calvin and OCC. Element *lock* can be either an exclusive lock (EX) or a shared lock (SH). In HDCC, if a Calvin sub-transaction cannot acquire the lock on x, it waits until the lock is released. For an OCC sub-transaction, it requests EX locks in the prepare phase. If any of the locks is held by other sub-transactions, it aborts the whole transaction. An OCC sub-transaction releases *lock* only after it commits; $x.wid$ maintains the $ID.TID$ of the most recent sub-transaction that modifies x. A Calvin sub-transaction updates $x.wid$ to its transaction $ID.TID$ after it writes x. An OCC sub-transaction updates $x.wid$ during its commit phase. In the execution phase, it stores $x.wid$ into its read set when it reads x. In the prepare phase, it checks whether $x.wid$ has been modified by any other sub-transactions after its first read operation on x. If $x.wid$ has been modified, it aborts; otherwise, it checks whether the lock on x is held by any Calvin sub-transaction. If it is, the sub-transaction aborts the whole transaction.

Algorithm 1 demonstrated the process of OCC validation. For each OCC sub-transaction T^o_{i-k}, its prepare phase is extended with two steps. The first step is to request locks on data items in the write set of T^o_{i-k}, described in Function GET-LOCK-O (line 1–4) of Algorithm 1. T^o_{i-k} aborts the whole transaction if any request fails (line 3). The second step is to perform the local validation, shown in Function VALIDATION-LOCAL-O (line 5–8). If any data item in the read set of T^o_{i-k} has been modified by other Calvin or OCC sub-transactions, T^o_{i-k} aborts the whole transaction (line 7).

Example 2. *Figure 5 illustrates how the lock-sharing mechanism addresses the issue raised in Example 1. T^c_{1-1} first writes x and updates $x.wid$ to 1, which is $T^c_{1-1}.ID.TID$ (step ①). Then, T^o_{3-1} reads x with the value written by T^c_{1-1} (step ②), writes the new value 3 of x in its write set. Next, T^c_{1-1} commits and releases the lock on x. Subsequently, T^o_{3-1} enters the prepare phase, in which it acquires the lock on x, and passes the validation (step ③) because x has not been modified by any other transaction since its first read on x. During the validation, T^c_{2-1} requests the lock on x, and waits for T^o_{3-1} to release the lock (step ④). After T^o_{3-1} commits, T^c_{2-1} acquires the lock on x and writes 2 to x. The above schedule is serializable and the equivalent serializable order is $T^c_{1-1} \rightarrow T^o_{3-1} \rightarrow T^c_{2-1}$.* □

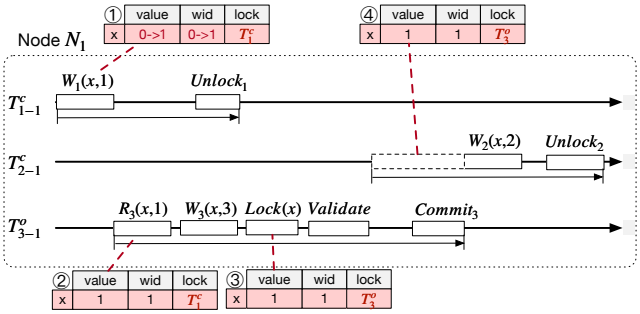


Figure 5: A running example of lock-sharing mechanism

Since a separate schedule of transactions by either Calvin or OCC satisfies conflict serializability, we now prove that a schedule of any two concurrent sub-transactions on the same node using HDCC, with one Calvin transaction, and the other OCC transaction, also ensures conflict serializability. Let $C(T_i)$ be the point in time at which transaction T_i commits. A transaction T_i is said to commit before another transaction T_j if $C(T_i) < C(T_j)$. In HDCC, if T_{i-k} is a Calvin sub-transaction, then the commit time $C(T_{i-k})$ of T_{i-k} is set to the point in time at which T_{i-k} starts to release its locks. If T_{i-k} is an OCC sub-transaction, $C(T_{i-k})$ is set to the point in time at which T_{i-k} enters the commit phase. *Thus, each sub-transaction has one and only one commit time.*

Theorem 1. *In HDCC, given two concurrent sub-transactions T^c_{i-k} and T^o_{j-k} executed on the same node, if $T^c_{i-k} \rightarrow T^o_{j-k}$ (resp. $T^o_{j-k} \rightarrow T^c_{i-k}$), $C(T^c_{i-k}) < C(T^o_{j-k})$ (resp. $C(T^o_{j-k}) < C(T^c_{i-k})$).* □

Proof. T^c_{i-k} and T^o_{j-k} must be in conflict; otherwise, the two sub-transactions do not have any dependence. Without loss of generality, let T^c_{i-k} and T^o_{j-k} be conflict on data item x. Because the partial order of T^c_{i-k} and T^o_{j-k} can only be $T^c_{i-k} \rightarrow T^o_{j-k}$ or $T^o_{j-k} \rightarrow T^c_{i-k}$. In both cases, at least one transaction performs a write operation on x, resulting in a read-write, write-write, or write-read dependency. We then prove the two cases individually. □

Case 1: $T^c_{i-k} \rightarrow T^o_{j-k}$. (1) T^c_{i-k} first reads x, and T^o_{j-k} then writes x. When the lock has not released by T^c_{i-k}, T^o_{j-k} detects the conflict by the lock and aborts; otherwise, T^o_{j-k} cannot detect the conflict, and continues the execution, which still satisfies $C(T^c_{i-k}) < C(T^o_{j-k})$. (2) T^c_{i-k} first write x, and T^o_{j-k} then writes x. It is the same as (1). (3) T^c_{i-k} first writes x, and T^o_{j-k} then reads x. When the EX lock has not released by T^c_{i-k}, T^o_{j-k} detects the conflict by the lock and aborts; otherwise, T^o_{j-k} cannot detect the conflict, and continues the execution, which satisfies $C(T^c_{i-k}) < C(T^o_{j-k})$ as well.

Case 2: $T^o_{j-k} \rightarrow T^c_{i-k}$. (1) T^o_{j-k} first reads x, and T^c_{i-k} then writes x. Because T^o_{j-k} does not add lock on x, T^c_{i-k} cannot detect the read-write conflict. HDCC solves this problem in the local validation of T^o_{j-k}. If T^o_{j-k} detects the conflict that there exists an EX lock on x or $x.wid$ that has been modified, T^o_{j-k} aborts; otherwise, T^o_{j-k} continues execution. This case satisfies $C(T^o_{j-k}) < C(T^c_{i-k})$. (2) T^o_{j-k}

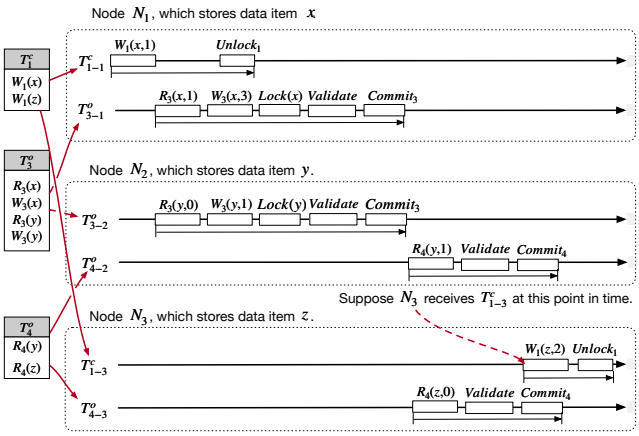


Figure 6: A running example of incorrect global schedule

first writes x, and T^c_{i-k} then writes x. When the EX lock has not released by T^o_{j-k}, T^c_{i-k} detects the conflict by the lock and waits until T^o_{j-k} commits; otherwise, T^c_{i-k} cannot detect the conflict because T^o_{j-k} has committed and released the lock. Clearly, they both satisfy $C(T^o_{j-k}) < C(T^c_{i-k})$. (3) T^o_{j-k} first writes x, and T^c_{i-k} then reads x. It is the same as (2). □

Theorem 2. *In HDCC, given two concurrent sub-transactions T^c_{i-k} and T^o_{j-k} executed on the same node, the schedule of these two sub-transactions satisfies conflict serializability.*

Proof. Suppose that the schedule is not conflict serializable, i.e., $T^c_{i-k} \rightarrow T^o_{j-k}$ and $T^o_{j-k} \rightarrow T^c_{i-k}$ are both true. According to Theorem 1, the commit order of T^c_{i-k} and T^o_{j-k} forms a cycle as well. However, each sub-transaction has one and only one commit time, making it impossible to form such a cycle. □

Discussion. (1) No deadlocks exist in HDCC. This is because HDCC allows a Calvin transaction to wait for locks held by OCC transactions, but does not allow an OCC transaction to wait for locks held by Calvin transactions. (2) Compared with Snapper, HDCC achieves a higher concurrency. Consider the three sub-transactions T_{1-1}, T_{2-1}, and T_{3-1} in Example 2. HDCC allows T_{3-1} to commit. However, in Snapper, T_{3-1} cannot commit, because Snapper treats batched transactions T_{1-1} and T_{2-1} as a whole and forms two reversed dependencies $B_1 \rightarrow T_{3-1}$ and $T_{3-1} \rightarrow B_1$.

4.2 The global validation mechanism

Given a set of transactions, the lock-sharing mechanism in HDCC guarantees a serializable schedule of their sub-transactions on each node. However, as shown in Example 3, it cannot ensure that the serializable order of transactions is consistent across nodes.

Example 3. *In Figure 6, T^c_1 updates x and z on nodes N_1 and N_3, respectively; T^o_3 updates x and y on nodes N_1 and N_2, respectively; T^o_4 reads y and z on nodes N_2 and N_3, respectively. On N_1, T^c_{1-1} updates x before T^o_{3-1} does, forming the dependency $T_1 \rightarrow T_3$. On N_2, T^o_{3-2} updates y before T^o_{4-2} reads it, forming the dependency $T_3 \rightarrow T_4$. Additionally, on N_3, T^o_{4-3} reads z before T^c_{1-3} updates it, forming the dependency $T_4 \rightarrow T_1$. Consequently, the dependencies among T_1, T_3, and T_4 form a cycle: $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$. This indicates that the schedule of these transactions is not conflict-serializable.* □

To prevent forming dependency cycles between Calvin and OCC transactions, we propose the global validation mechanism. This mechanism attempts to obtain $T^o_j.DS^c$, defined in Definition 1, for each OCC transaction T^o_j, and do the global validation over transactions of $T^o_j.DS^c$ upon the commit of T^o_j. If there exists a transaction $T^c_i \in T^o_j.DS^c$ that has not yet committed, meaning that the committed order between T^c_i and T^o_j is not consistent with the dependency order between them, according to Theorem 1, T^o_j needs to abort; otherwise, T^o_j is able to commit.

Definition 1 (Dependent Set). *Given an OCC transaction T^o_j, the dependent set of T^o_j, denoted as $T^o_j.DS$, is defined as the set of transactions that T^o_j depends on. Among them, the dependent OCC transactions set $T^o_j.DS^o$ is defined as $\{T^o_i | T^o_i \in T^o_j.DS\}$, and the dependent Calvin transactions set $T^o_j.DS^c$ is defined as $\{T^c_i | T^c_i \in T^o_j.DS\}$.* □

Definition 2 (Direct Dependent Set). *The direct dependent set of T^o_j, denoted as $T^o_j.DDS$, is defined as the transactions of $T^o_j.DS$, each T of which takes conflicting access to the same data items with T^o_j, and $T \rightarrow T^o_j$. Among them, the dependent OCC transactions set $T^o_j.DDS^o$ is defined as $\{T^o_i | T^o_i \in T^o_j.DDS\}$, and the dependent Calvin transactions set $T^o_j.DDS^c$ is defined as $\{T^c_i | T^c_i \in T^o_j.DDS\}$.* □

Definition 3 (Indirect Dependent Set). *The indirect dependent set of T^o_j, denoted as $T^o_j.IDS$, is defined as the difference between $T^o_j.DS$ and $T^o_j.DDS$, i.e., $T^o_j.DS - T^o_j.DDS$. Among them, the OCC transaction set $T^o_j.IDS^o$ is $\{T^o_i | T^o_i \in T^o_j.IDS\}$, and the Calvin transaction set $T^o_j.IDS^c$ is $\{T^c_i | T^c_i \in T^o_j.IDS\}$. According to transitive closure, we can have: $T^o_j.IDS = \bigcup_{T_i \in T^o_j.DDS}\{T_i.DS\}$.* □

According to Definition 2 and Definition 3, $T^o_j.DS^c$ is the union of set $T^o_j.DDS^c$ and set $T^o_j.IDS^c$. Acquiring $T^o_j.IDS^c$ requires computing the transitive closure of the indirect dependent set of each transaction in $T^o_j.DDS^c$. When the set $T^o_j.DDS^c$ is relatively large, this computation becomes prohibitively expensive.

Definition 4 (Maximal Calvin Transaction ID). *Given a transaction set S, the maximal Calvin transaction ID of S, denoted as $S.MaxCid$, is defined as the maximal ID of Calvin transactions in S, i.e., $S.MaxCid = \max\{T^c_i.ID | T^c_i \in S\}$.* □

Definition 5 (Maximal Dependent Transaction ID). *The maximal dependent transaction ID of an OCC transaction T^o_j, denoted as $T^o_j.gMaxCid$, is defined as the maximal Calvin transaction ID of $T^o_j.DS^c$, which is $T^o_j.DS^c.MaxCid$.* □

Definition 6 (Extended Dependent Set). *The extended dependent set of T^o_j, which denoted as $T^o_j.EDS^c$, is defined as: $\{T^c_i | T^c_i.ID \leq T^o_j.gMaxCid\}$.* □

As previously mentioned, computing $T^o_j.DS^c$ for T^o_j is expensive. Instead, we attempt to compute a superset of $T^o_j.DS^c$, which is relatively lightweight. Upon the commit of T^o_j, if all transactions

in $T_j^o.DS^c$ have committed, then T_j^o is ready to commit; otherwise, T_j^o aborts. To this end, we propose the concept of the maximal dependent transaction ID $T_j^o.gMaxCid$ for T_j^o, as defined in Definition 5, and use $T_j^o.gMaxCid$ to define the superset $T_j^o.EDS^c$ of $T_j^o.DS^c$ in Definition 6. Recall that, given two Calvin transactions T_k^c and T_l^c with $T_k^c.ID < T_l^c.ID$, T_k^c always acquires the locks on the co-accessed data items earlier than T_l^c, and T_l^c commits after T_k^c. Since $T_j^o.EDS^c$ includes the Calvin transaction T with the largest ID in $T_j^o.DS^c$, and all other Calvin transactions with IDs less than or equal to $T.ID$, obviously, $T_j^o.EDS^c$ is a superset of $T_j^o.DS^c$.

Theorem 3. *$\forall T_j^o$, $T_j^o.EDS^c$ is a superset of $T_j^o.DS^c$.* □

Formula 1 shows how to compute $T_j^o.gMaxCid$. We omit the discussion of Equation 1–3, which are self-explanatory. We now discuss how to transform Equation 4 to 5. Recall in Calvin, for any transaction T_m^c, transactions in $T_m^c.DS^c$ must be scheduled to execute before T_m^c. Thus, we have $T_m^c.DS^c.MaxCid < T_m^c.ID$. Since $T_m^c \in T_j^o.DDS^c$, we have $T_m^c.ID < T_j^o.DDS^c.MaxCid$. Therefore, $\{T_m^c.DS^c.MaxCid \mid T_m^c \in T_j^o.DDS^c\}$ in Equation 4 can be omitted.

Formula 1 (Equivalence transformation formula).

$$T_j^o.gMaxCid = T_j^o.DS^c.MaxCid \quad (1)$$

$$= \max(T_j^o.DDS^c.MaxCid, T_j^o.IDS^c.MaxCid) \quad (2)$$

$$= \max(T_j^o.DDS^c.MaxCid, \{T_m.DS^c.MaxCid | T_m \in T_j^o.DDS\} \quad (3)$$

$$= \max(T_j^o.DDS^c.MaxCid, \{T_m^c.DS^c.MaxCid | T_m^c \in T_j^o.DDS^c\}, \{T_n^o.DS^c.MaxCid | T_n^o \in T_j^o.DDS^o\}) \quad (4)$$

$$= \max(T_j^o.DDS^c.MaxCid, \{T_n^o.gMaxCid | T_n^o \in T_j^o.DDS^o\}) \quad (5)$$

For the implementation, we introduce two elements *cid* and *cid′* associated with each data item x in the lock table. $x.cid$ and $x.cid'$ are used to compute $T_j^o.DDS^c.MaxCid$ and $\max(\{T_n^o.gMaxCid | T_n^o \in T_j^o.DDS^o\})$ in Equation 5 of Formula 1, respectively. Upon any read/write of a Calvin transaction T_i^c, T_i^c would update $x.cid$ properly if $T_i^c.ID > x.cid$. Thus, $x.cid$ always maintains the current maximal Calvin transaction ID on x. In the validation phase of any OCC transaction T_k^o, T_k^o collects the *cid* and *cid′* of each data item that it has ever accessed, and calculates its *gMaxCid* by computing the maximum value of these *cid* and *cid′*. Upon the commit of T_k^o, for each data item x that it has ever accessed, T_k^o sets $x.cid'$ to $\max\{x.cid', T_k^o.gMaxCid\}$. In the prepare phase, T_j^o conducts global validation, which checks whether all the Calvin sub-transactions with IDs smaller or equal to $T_j^o.gMaxCid$ have been committed at each node accessed by T_j^o. If successful, T_j^o commits; otherwise, it aborts.

Example 4. *Continuing Example 3, Figure 7 shows how the global validation mechanism prevents the dependency cycles. In step ①, when T_{1-1}^c writes x, it sets $x.cid$ to $T_1^c.ID$, which is $\langle 1,1\rangle$. In step ②, T_3^o conducts the validation phase, in which it collects $x.cid$, $x.cid'$, $y.cid$, and $y.cid'$. Next, T_3^o computes $T_3^o.DDS^c.MaxCid$, which is $\langle 1,1\rangle$, and $max\{T_n^o.gMaxCid | T_n^o \in T_3^o.DDS^o\}$, which is $\langle 0,0\rangle$. According to Equation 5, $T_3^o.gMaxCid$ is set to $\langle 1,1\rangle$. In step ③, upon T_3^o commits, it updates $x.cid'$ and $y.cid'$ to $T_3^o.gMaxCid$, which is $\langle 1,1\rangle$. In step ④, following the same procedure as T_3^o, T_4^o calculates its gMaxCid, which is $\langle 1,1\rangle$. Then, T_4^o detects that transaction T_1^c with its ID equal to $T_4^o.gMaxCid$ on N_3 has not committed, and then T_4^o aborts, breaking the dependency cycle $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$.* □

Algorithm 2: Global validation for OCC

```
1  function VALIDATE-GLOBAL-O():
2      // the first round of validation
3      foreach stxn in sub-transactions do
4          ⟨result, lMaxCid⟩ := RPC_stxn ::GET-LOCK-O();
5          if result == ABORT then return ABORT;
6          gMaxCid := max(gMaxCid, lMaxCid);
7      end
8      // the second round of validation
9      foreach stxn in sub-transactions do
10         result := RPC_stxn ::VALIDATE-LOCAL-O(gMaxCid);
11         if result == ABORT then return ABORT;
12     end
13 end
14 function GET-LMAXCID():
15     foreach x in rs ∪ ws do
16         lMaxCid := max{lMaxCid, x.cid, x.cid'};
17     end
18     return lMaxCid;
19 end
20 function VALIDATE-HYBRID(gMaxCid):
21     if maxBID < gMaxCid.BID then return ABORT ;
22     foreach txn in uncommitted Calvin Txn do
23         if txn.ID ≤ gMaxCid.ID then return ABORT ;
24     end
25     return COMMIT;
26 end
```

Algorithm 2 illustrates the global validation mechanism, which consists of two-round communications. In the first round, the coordinator of T_j^o sends a remote procedure call (RPC_{stxn}) to each sub-transaction *stxn* of T_j^o and triggers *stxn* to request locks as discussed in Section 4.1 (line 3,4). In Function GET-LOCK-O, *stxn* invokes Function GET-LMAXCID (line 14–19) to calculate a local maximum value among *cid* and *cid′* of all data items it accesses, denoted as *lMaxCid* (line 15–16). Subsequently, the coordinator collects *lMaxCid* of each sub-transaction and calculates the *gMaxCid* (line 6). In the second round, the coordinator issues another RPC to notify *stxn* of T_j^o to perform the local validation scheme that examines whether there are any sub-transaction with its *ID* smaller than or equal to $T_j^o.gMaxCid$ that has not committed (line 9–12). In Function VALIDATE-LOCAL-O, *stxn* invokes Function VALIDATE-HYBRID (line 20–26 in Algorithm 2 and line 11 in Algorithm 1). If there is any uncommitted or unreceived Calvin transaction with its ID smaller than or equal to $T_j^o.gMaxCid$, T_j^o aborts (line 22–23).

Now we prove that the schedule of transactions using HDCC is conflict serializable in Theorem 4, and Theorem 5.

Theorem 4. *In HDCC, given any two committed transactions T_i^c and T_j^o, on each node N_k where there exists a dependency between T_{i-k}^c and T_{j-k}^o, T_{i-k}^c and T_{j-k}^o must have a consistent order of either $C(T_{i-k}^c) < C(T_{j-k}^o)$ or $C(T_{j-k}^o) < C(T_{i-k}^c)$.* □

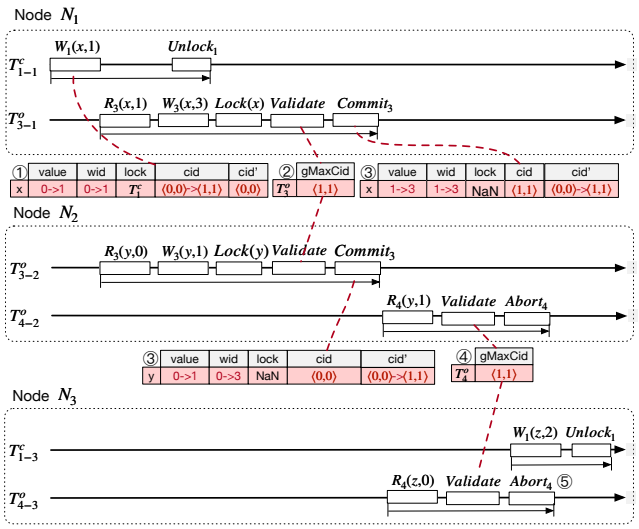


Figure 7: A running example of global validation

Proof. We shall prove by contradiction that there exist $C(T^c_{i-k}) < C(T^o_{j-k})$ on node N_k and $C(T^o_{j-l}) < C(T^c_{i-l})$ on node N_l. According to Theorem 1, it must be the case that $T^c_{i-k} \rightarrow T^o_{j-k}$ and $T^o_{j-l} \rightarrow T^c_{i-l}$. However, $T^c_{i-k} \rightarrow T^o_{j-k}$ indicates that $T_i.ID \leq T^o_j.gMaxCid$ and thus T^o_j would abort because T^c_{i-l} has not committed. This contradicts the assumption that both transactions could commit. □

Theorem 5. *In HDCC, the schedule of any two concurrent transactions T^c_i and T^o_j is conflict serializable.* □

Proof. If T^c_i and T^o_j do not have dependencies, their schedule is obviously conflict-serializable. Now, we shall prove by contradiction that if they do have dependencies, these dependencies form a cycle. According to Theorem 2, any two transactions cannot form a cycle on any single node. Thus, there must exist two nodes, N_k and N_l, with $T^c_{i-k} \rightarrow T^o_{j-k}$ on N_k and $T^o_{j-l} \rightarrow T^c_{i-l}$ on N_l. According to Theorem 1, we can conclude that these two transactions have $C(T^c_{i-k}) < C(T^o_{j-k})$ and $C(T^o_{j-l}) < C(T^c_{i-l})$. However, according to Theorem 4, if both T^c_i and T^o_j commit, their dependencies cannot form a cycle. This contradicts our previous assumption. □

4.3 Optimizations

In this section, we first introduce the rule-based assignment mechanism, aiming to select the best concurrency control algorithm. We then present two additional optimizations for HDCC.

4.3.1 The rule-based assignment. In this assignment, the first rule is that, for a transaction T without pre-declaring its read/write set, T is scheduled by OCC.

Rule 1. *Given a transaction T without pre-declaring its read/write set, T is scheduled by OCC.* □

As opposed to the basic assignment, for a transaction T that has declared read/write set, we do not simply schedule T using Calvin. Instead, we make the adaptive assignment that carefully considers T's characteristics. As discussed before, Calvin excels in high contention workloads or distributed transactions, while OCC performs better in either low or medium contention workloads. We then propose the following two heuristic rules, i.e., Rule 2 and 3, to do the concurrency control algorithm assignment.

Rule 2. *Given a transaction T that has declared read/write set, if T is a distributed transaction, T is scheduled by Calvin.* □

To harness the performance advantages of concurrency control algorithms across various contention workloads, HDCC determines the contention level of workloads on each partition by measuring the number of conflicts. Given a partition P_x, let $C.P_x$ be the number of Calvin transactions that wait for the lock on data item x that resides in P_x, and $O.P_x$ be the number of OCC transactions that abort due to the failed validation on x. Both $C.P_x$ and $O.P_x$ are stored in P_x. A partition P_x is considered to be *hot*, if the sum of $C.P_x$ and $O.P_x$ exceeds a certain threshold during a time period.

Rule 3. *Given a transaction T that has declared read/write set, if T reads/writes hot partitions, T is scheduled by Calvin.* □

If T has declared read/write set, but is not a distributed transaction or does not operate data items in the hot partition, we schedule T using OCC.

Rule 4. *Given a transaction T that has declared read/write set, if T is neither a distributed transaction, nor reads/writes hot partitions, T is scheduled by OCC.* □

4.3.2 Re-schedule of aborted OCC transactions. At the point in time that an OCC transaction T^o_j aborts due to the conflicts, we have collected the complete read/write set of T^o_j, which can be seen as a reconnaissance run for Calvin algorithm. Consequently, the analyzer can reassign the algorithm according to the rules outlined in Section 4.3.1 based on T^o_j's read/write set. Note, values of the data items in the read/write set of restarted transaction T^o_j may change, causing T^o_j to repeatedly abort. However, as reported in [42], this rare happens.

4.3.3 Immediate read and deferred commit. Consider the case that an OCC transaction T^o_j reads data item x that has been written by a Calvin transaction T^c_i. This case produces two possible schedules. The first schedule is that T^c_i commits before T^o_j. In this schedule, T^o_j can commit as discussed in Section 4.1. The second schedule is that T^o_j enters the prepare phase before T^c_i commits. In the original design, T^o_j could detect that x is locked by T^c_i, and aborts. However, as Calvin transactions seldom abort, we defer the decision instead of immediate abort, so as to reduce the false positive aborts. To do this, in the prepare phase, we defer the commit of an OCC transaction T^o_j if all the following conditions satisfy: (1) at least one data item x in the read set has been locked by a Calvin transaction T^c_i; (2) $x.wid$ remains unmodified after the first read operation of T^o_j on x; and (3) $x.wid$ equals to $T^c_i.ID$. If T^o_j is deferred, it records the ID of T^c_i and waits for the commit of T^c_i. If T^c_i aborts, T^o_j undergoes a cascading abort; otherwise, T^o_j commits.

5 FAILURE RECOVERY

In this section, we first give the general idea, then present the two-log-interleaving mechanism, and finally discuss its correctness.

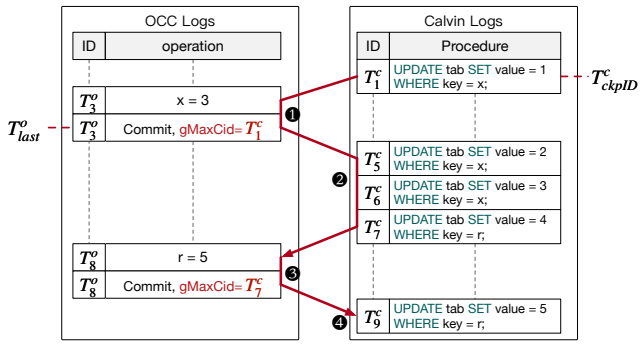


Figure 8: Two-log-interleaving mechanism

5.1 General idea

Upon any system failure, we need to perform redo and undo operations to ensure data consistency. The redo operation follows the serializable orders of committed transactions to recover the writes of them. The undo operation rolls back the modification of transactions that are not yet committed before the failure.

In HDCC, we do not need to perform any undo operations. This is because we only flush data written by committed transactions to disk; data from uncommitted transactions are never flushed. In our design, HDCC periodically creates checkpoints. During each checkpointing, for Calvin transactions, we select a transaction T^c_{ckpID} and flush the data items written by the transactions with IDs less than or equal to $T^c_{ckpID}.ID$. Note that Calvin does not write dirty pages, but flushes data items to disk. Since transactions with IDs greater than $T^c_{ckpID}.ID$ may have writes, we do not flush these writes. Instead, we maintain these writes as copies. Details of checkpointing the Calvin transactions are discussed in Section 2.1. For OCC transactions, we stop receiving new transactions, wait for the ongoing transactions to complete, and then flush the data to disk. The last committed OCC transaction is recorded as T^o_{last} for failure recovery. After the checkpointing process is complete, HDCC restarts to accept and execute new OCC transactions.

In HDCC, we need to perform redo operations. Unlike traditional databases that have only a single category of logs, HDCC maintains both OCC logs and Calvin logs. Because OCC logs and Calvin logs are separate, the commit order between OCC transactions and Calvin transactions is lost. Therefore, to perform redo operations, we first need to recover the commit order between OCC transactions and Calvin transactions, and then replay the logs according to the ascending commit order of transactions.

5.2 Two-log-interleaving mechanism

HDCC has two types of logs, Calvin logs and OCC logs, as shown in Figure 8. Calvin logs record transactions' logical logs, which contain their IDs, and their procedures. For example, transaction T^c_1 records its procedure like "UPDATE tab SET value = 1 WHERE key = x". OCC logs record the redo logs and commit logs of OCC transactions. Redo logs record the new value of the data items modified by the committed OCC transactions, while commit logs indicate that OCC transactions have successfully committed. Besides, we maintain $T^o.gMaxCid$ in the commit log of T^o. Recall in Section 4.2, $T^o.gMaxCid$ represents the maximum ID of those dependent Calvin transactions that commit before T^o. As a result, by maintaining $gMaxCid$, we capture the commit order between OCC transactions and Calvin transactions. As shown in Figure 8, T^o_3 records its redo log as $\langle T^o_3 : x = 3\rangle$, and its commit log as $\langle T^o_3 : Commit, gMaxCid = T^c_1\rangle$, indicating that T^o_3 needs to be recovered after T^c_1.

On any node N, upon a system failure, after N restarts, our two-log-interleaving mechanism follows an interleave manner to replay the logs of committed transactions. (1) We sequentially scan the OCC logs to identify the first OCC transaction T^o_j ordered after T^o_{last} that needs to be recovered. (2) We sequentially scan Calvin logs and replay the logs of all Calvin transactions with their IDs smaller than or equal to $T^o_j.gMaxCid$, but greater than $T_{ckpID}.ID$. (3) We replay the OCC logs of transaction T^o_j, and find the next OCC transaction T^o_{j+1} in OCC logs. (4) We continue to scan Calvin logs and replay each Calvin transaction T^c_i where $T^c_i.ID$ falls within the range $T^o_j.gMaxCid < T^c_i.ID \leq T^o_{j+1}.gMaxCid$. (5) We repeat step 3 to 4 until all OCC transactions involved in the OCC logs have been recovered. (6) We restore the other Calvin transactions by replaying the rest of Calvin logs.

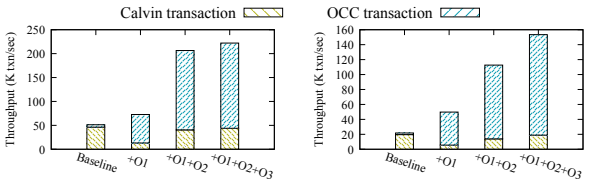
Example 5. *As depicted in Figure 8, the solid red lines indicate the recovery sequence of logs in HDCC upon a failure. Due to a checkpoint created beforehand, HDCC can recover from the two transactions T^c_{ckpID} (T^c_1), and T^o_{last} (T^o_3). HDCC first sequentially scans the OCC logs to find the first OCC transaction T^o_8 that needs to be recovered (step ❶). Since the $T^o_8.gMaxCid$ is T^c_7, meaning that HDCC needs to recover T^c_7 before recovering T^o_8. Thus, HDCC sequentially scans Calvin logs and replays the Calvin transactions from T^c_5 to T^c_7 (step ❷). Subsequently, HDCC recovers the transaction T^o_8. Upon completion, HDCC finds all OCC transactions involved in OCC logs have been recovered (step ❸). At this stage, HDCC restores the other Calvin transaction T^c_9 by replaying its Calvin log (step ❹).*

Note that in the original Calvin logging mechanism, the locally recorded logs are incomplete because Calvin records the transactions received by nodes, rather than the transactions executed by nodes. This means that when a node undergoes failure recovery, the node may lack the logs of transactions that need to be executed on this node. To address these issues, HDCC modifies Calvin's logging mechanism. Upon receiving a transaction, each node will persist the received transaction, ensuring that each node maintains a complete set of transactions that need to be executed on that node. Furthermore, during the redo of a local Calvin transaction T^c_i, its writes may rely on the data read from other nodes, which could have already been overwritten by the next batches. To address this issue, HDCC introduces Calvin redo logs. When a Calvin transaction's writes depend on a remote read, the outcome of that write operation is logged in the Calvin redo logs.

5.3 Correctness

We prove the correctness of our failure recovery in Theorem 6 and provide its TLA+ specification in the appendix.

Theorem 6. *In HDCC, the transaction schedule for failure recovery is conflict equivalent to that of the original execution.* □



(a) Effect of Optimization (YCSB) (b) Effect of Optimization (TPC-C)

Figure 9: Optimization analysis

Proof. According to our global validation mechanism, $T_j^o.gMaxCid$ indicates the order between Calvin and OCC transactions. The recorded order in the log aligns with the original execution sequence. For transactions without dependencies, conflicts do not exist; hence, their order in the log is always conflict equivalent to their original execution order. Therefore, the transaction order in the log is conflict equivalent to the original execution order. Second, during the redo operations of HDCC, both Calvin transactions and OCC transactions are replayed according to the order recorded in the log. Consequently, the transaction schedule order during failure recovery aligns with the order recorded in the log.

□

6 EVALUATION

In this section, we conduct a comprehensive experimental evaluation of HDCC and the following research questions are addressed:

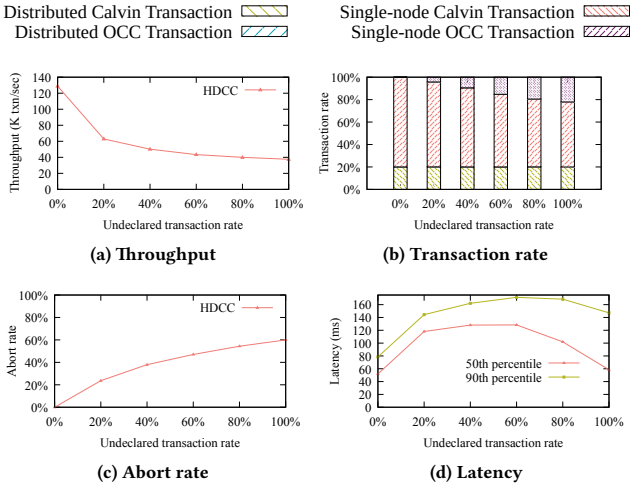
(1) How effective are the optimizations, the performance under dynamic workloads, and the scalability, including the logging overhead, in HDCC? (Section 6.2)
(2) How does HDCC perform compared with its constituent algorithms (i.e., Calvin and OCC) and the state-of-the-art algorithms (e.g., Aria and Snapper)? (Section 6.3)

6.1 Experimental setup

Test platform. We deploy a share-nothing in-memory database system with 2 servers, each equipped with an 18-core (36-thread) 2.20GHz Intel Xeon Gold processor 5220 and 156 GB memory. Each server installs a client and a database. On the database side, we use 16 worker threads, 1 sequencer thread, 1 scheduler thread, 1 analyzer thread, and some other threads that are responsible for inter-node communication and transaction management. On the client side, there are four worker threads responsible for generating transactions and also some threads responsible for inter-node communication. Note, in the scalability experiments conducted in Section 6.2.3, we deploy the system on 12 virtual nodes, each node equipped with 16 threads and 32 GB memory.

Workloads. Our experiments are conducted with two benchmarks, namely YCSB [11] and TPC-C [4]. For YCSB, we generate about 8 million records for each database node and each transaction executes 10 read/write operations that access data items following the Zipfian distribution, with a default skew factor of 0.9 and a write-read rate of 0.2 (i.e., 20% writes and 80% reads). For TPC-C, we generate 32 warehouses for each database node and exclusively employ NewOrder and Payment transactions, as the remaining transaction types are single-node transactions that are not suitable for our evaluation and they account for a smaller proportion.

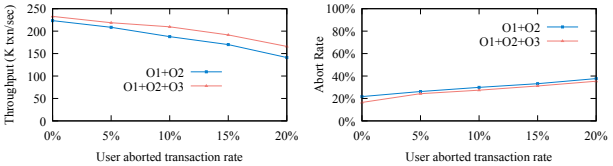
Configurations. In addition, we introduced two metrics, i.e., the distributed transaction rate, and the undeclared transaction rate.



(a) Throughput (b) Transaction rate

(c) Abort rate (d) Latency

Figure 10: The overhead analysis of $O2$



(a) Throughput (b) Abort rate

Figure 11: The overhead analysis of $O3$

The distributed transaction rate quantifies the proportion of transactions that access data across multiple nodes, while the undeclared transaction rate measures the proportion of transactions that lack the knowledge of their read/write sets. By default, we configure the distributed transaction rate to 20% for the YCSB workload. For the TPC-C workload, we adhere to its default values and set the distributed transaction rate to 10% for NewOrder transactions and 15% for Payment transactions. Unless otherwise stated, we set the undeclared transaction rate to 0.

6.2 The effeciency of HDCC

6.2.1 Effect of optimizations. This part evaluates the effectiveness of the three optimizations, namely ***O1*** (rule-based assignment), ***O2*** (re-scheduling of aborted OCC transactions), and ***O3*** (immediate read and deferred commit). By default, we set the undeclared transaction rate to 10% for both YCSB and TPC-C benchmarks.

Figure 9a shows the throughput of different optimizations on the YCSB benchmark. It is evident that, optimization *O1*, *O1+O2*, and *O1+O2+O3* improve the throughput 0.4×, 3.0×, and 3.3×, respectively, compared to basic assignment utilized in Snapper (labeled ***Baseline***). The performance improvement introduced by *O1* can be attributed to its intelligent transaction assignment based on their characteristics, effectively capitalizing on the benefits of OCC in low contention scenarios and Calvin in distributed settings. The performance enhancement brought by *O2* is a result of its capability to maintain the read/write sets of aborted OCC transactions, enabling HDCC to potentially assign Calvin to these aborted transactions, thus optimizing the system. The performance boost associated with *O3* is due to its ability to allow OCC transactions

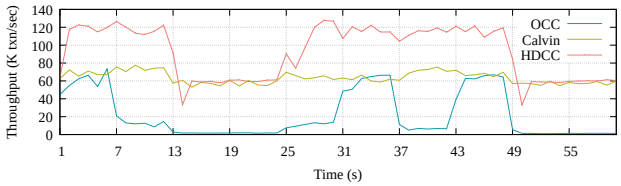


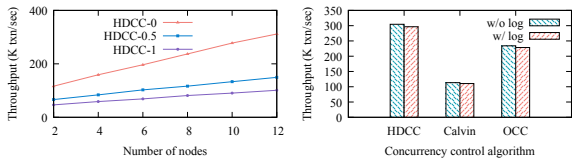
Figure 12: Effect of dynamic workload (YCSB)

to read dirty data items written by uncommitted Calvin transactions, thereby further reducing the abort rate and enhancing concurrency. Figure 9b shows the results of different optimizations on the TPC-C benchmark. The trends depicted in the figure align with the behavior observed under the YCSB workload. Optimization *O1*, *O1+O2*, and *O1+O2+O3* have improved the throughput compared to the baseline by factors of 1.3 ×, 4.2×, and 6.0×, respectively.

Regarding the *O2* optimization, which contributes the most to the throughput improvement, we conducted an additional evaluation to test its overhead. Figure 10 illustrates the results under a high-conflict scenario (skew factor = 1.3) as the undeclared transaction rate varies from 0 to 100%. When the undeclared transaction rate is 0%, meaning all transactions are directly scheduled by Calvin, the transaction latency is minimal, approximately 50 ms at the median percentile. As the undeclared transaction rate increases, the throughput experiences a decline, but it eventually stabilizes at around 40 K transactions per second. On the contrary, compared to the scenario where the undeclared transaction rate is 0%, the median and 90th percentile latencies increased by up to 2.2×. We consider this additional overhead to be acceptable, as *O2* still ensures the maintenance of relatively high performance in scenarios with a higher undeclared transaction rate. Notably, when the undeclared transaction rate exceeds 60%, both the median and 90th percentile latencies decrease. As shown in Figure 10b, in such scenarios, a significant portion of single-node OCC transactions are committed, which results in a lower overall latency.

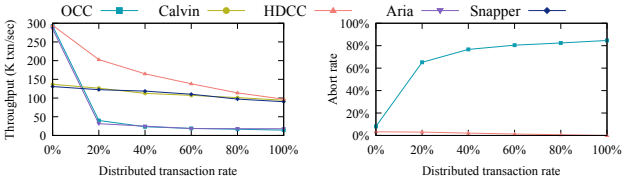
As described in Section 4.3.3, OCC transactions may experience cascading aborts when using *O3*. Thus, we conducted evaluations specifically focused on scenarios where Calvin transactions could potentially abort, as shown in Figure 11. By varying the proportion of transactions that are aborted by users from 0% to 20%, we observed a substantial 37% decrease in throughput for *O1+O2*. In contrast, *O1+O2+O3* exhibited a 29% decrease in throughput. This observation suggests that the throughput improvement introduced by *O3* surpasses the overhead incurred by cascading aborts. The abort rate depicted in Figure 11b further reinforces this observation, as the abort rate of *O1+O2+O3* is lower than that of *O1+O2*. For the following evaluations, HDCC adopts all optimizations.

6.2.2 Dynamic workload. To evaluate HDCC's adaptability to online dynamic workloads, we conduct a 60-second experiment, using the YCSB workload to simulate real-world workload scenarios. In particular, we switch the YCSB workload configurations every 6 seconds. During each switch, we randomly select a write-read rate from 0.1, 0.5, and 0.9, and a skew factor from 0.3, 0.9, and 1.5. Figure 12 shows the results, where switches occur at timestamps 7, 13, and 25, etc. We can observe that HDCC promptly responds and dynamically adjusts the algorithm assignment based on the changing workload. After each switch, the throughput of HDCC is rapidly adjusted to either better than or comparable to that of OCC or Calvin.



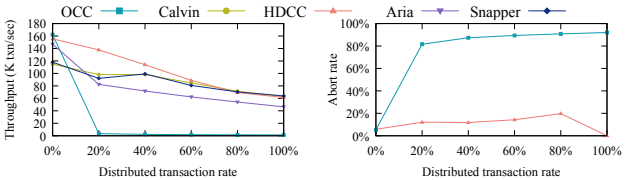
(a) Scalability **(b) Overhead of logging**

Figure 13: Scalability and overhead of logging (YCSB)



(a) Throughput **(b) Abort rate**

Figure 14: Effect of distributed transaction rate (YCSB)



(a) Throughput **(b) Abort rate**

Figure 15: Effect of distributed transaction rate (TPC-C)

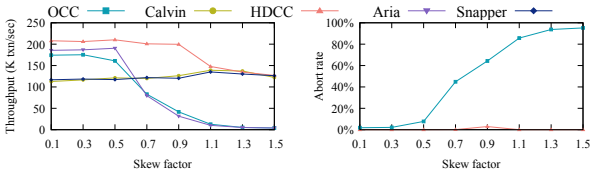
6.2.3 Scalability and logging overhead. We evaluate the scalability of HDCC using YCSB and TPC-C workloads by varying the number of nodes from 2 to 12. We report the scalability of HDCC under three different undeclared transaction rates with 0 (HDCC-0), 0.5 (HDCC-0.5), and 1 (HDCC-1). We only report the experimental results of YCSB due to similar findings. As shown in Figure 13a, the throughput of HDCC-0, HDCC-0.5, and HDCC-1 exhibit linear scalability, demonstrating excellent scalability characteristics.

As shown in Figure 13b, the logging mechanisms we implemented for durability incurred relatively minor and comparable overhead. With logging, there was a consistent and neglectable reduction in throughput for the OCC, Calvin, and HDCC (2.4%, 2.7%, and 2.7%, respectively). For fairness, we refrain from comparing the throughput with logging in Section 6.3 as Aria and Snapper have not specified logging mechanisms.

6.3 Compared with mainstream algorithms

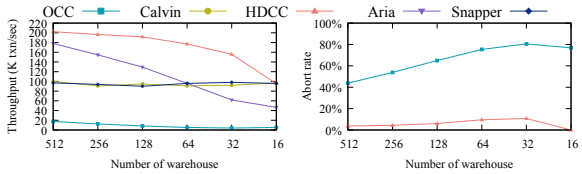
6.3.1 Effect of distributed transaction rate. As shown in Figure 14a, when the distributed transaction rate is 0, indicating all transactions are single-node transactions, HDCC, OCC, and Aria exhibit similar high throughput, surpassing Calvin and Snapper. In such scenarios, HDCC, Aria, and OCC can execute single-node transactions eliminating the need for the 2PC protocol, while Calvin and Snapper are hindered by the bottleneck of a single Scheduler.

As the distributed transaction rate increases, Calvin and Snapper maintain a relatively stable throughput, while Aria, OCC, and



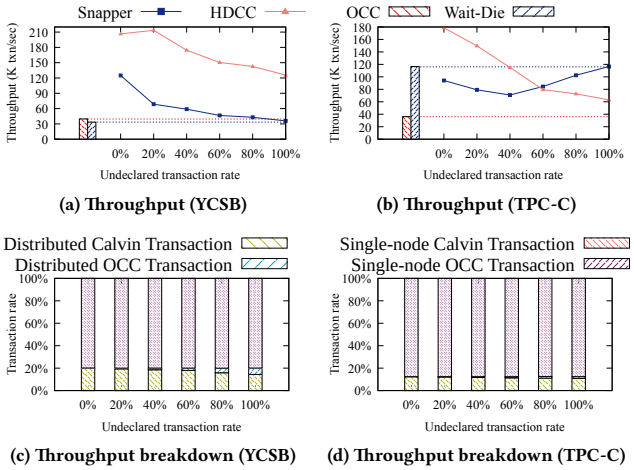
(a) Throughput (b) Abort rate

Figure 16: Effect of the contention level (YCSB)



(a) Throughput (b) Abort rate

Figure 17: Effect of the contention level (TPC-C)



(a) Throughput (YCSB) (b) Throughput (TPC-C)

(c) Throughput breakdown (YCSB) (d) Throughput breakdown (TPC-C)

Figure 18: Effect of undeclared transaction rate

HDCC experience a decreased throughput. Calvin and Snapper achieve comparable performance because when the undeclared transaction rate is set as the default value of 0, Snapper assigns all transactions to Calvin. HDCC, utilizing Calvin for distributed transactions through *O1*, exhibits a smaller decline in throughput. In contrast, OCC declines significantly due to the overhead of the 2PC protocol, resulting in longer execution time and a higher number of aborts for distributed transactions (as shown in Figure 14b). Aria exhibits a similar trend to OCC due to its implementation of a 2PC-like transaction processing mechanism, leading to performance levels on par with OCC. Note, that both Calvin and Aria are deterministic concurrency control, and in this experiment, Snapper assigns all transactions scheduled by Calvin, hence they do not have any abort rates in Figure 14b.

When the distributed transaction rate approaches 100%, the throughput of HDCC becomes comparable to that of Calvin, and is approximately 6.8× higher than that of OCC and Aria. At this stage, the majority of transactions in HDCC are scheduled using Calvin.

The results of the TPC-C benchmark, which share a similar trend to that of YCSB, are presented in Figure 15. However, Aria demonstrates significantly better performance than OCC under TPC-C. This enhancement can be attributed to our adherence to the experimental setups outlined in previous research [46], where we tailored the batch sizes of Aria differently for TPC-C and YCSB to maximize Aria's efficiency.

6.3.2 Effect of the contention level. Figure 16 shows the throughput and abort rate with the skew factor (SF) ranges from 0.1 to 1.5. Under low contention levels (SF <= 0.5), the performance of all five algorithms remains relatively stable. HDCC outperforms other algorithms due to the optimizations. Under moderate contention levels (0.5 < SF < 1.1), OCC and Aria are very sensitive to the contention and experience an increase in abort rate and a decline in throughput compared to the low contention scenario, while HDCC still maintains stable throughput mainly due to optimization *O2*. Under high contention levels (SF >= 1.1), almost all transactions in HDCC are scheduled by Calvin, leading to a similar throughput with Calvin. The performance of Snapper remains similar to that of Calvin.

Additionally, we conduct experiments on the TPC-C workload by varying the number of warehouses to simulate different contention levels (smaller number of warehouses indicating higher attention level). As shown in Figure 17, the trend of throughput for HDCC is similar to that of YCSB. Note that the throughput of OCC is notably poor in this setup due to the high contention imposed by the TPC-C workloads, which also leads to a high abort rate. Due to the same reason discussed in Section 6.3.1, Aria outperforms OCC.

6.3.3 Effect of undeclared transaction rate. This part compares the performance of HDCC with Snapper by varying the undeclared transaction rate ranging from 0 to 1.

Figure 18a shows that when all transactions pre-declare their read/write sets, HDCC performs significantly better than Snapper, achieving up to 3.1× higher throughput. This is because HDCC's *O1* assigns OCC to execute all local transactions even when transactions are declared and feasible for Calvin (refer to Figure 18c for throughput breakdown). On the contrary, Snapper does not consider workload characteristics and executes Calvin as long as all transactions are pre-declared. As the undeclared transaction rate increases, due to the *O2* and *O3* optimization, the throughput of HDCC experiences a slight decline, yet remains superior to Snapper. Figures 18b and 18d show the results on the TPC-C dataset, where HDCC exhibits a similar advantageous trend to that observed on the YCSB dataset, achieving up to 2.3× higher throughput. The exceptions are when the undeclared transaction rate is higher than 0.8, Snapper performs better than HDCC. This improvement can be mainly attributed to the fact that in the TPC-C scenario, the Wait-Die algorithm (used in Snapper) exhibits superior performance compared to OCC [20]. The higher efficiency of Wait-Die in handling conflicts leads to the overall better performance of Snapper in this specific scenario.

7 RELATED WORK

This section studies the related work of concurrency control algorithms and hybrid concurrency control algorithms.

Concurrency Control. Concurrency control algorithms can be typically divided into two categories: (1) deterministic algorithms, and (2) non-deterministic algorithms. The first category executes

transactions in a scheduled deterministic order. The classical algorithms like Calvin [42] make stringent assumptions that read/write sets of transactions must be pre-declared. Subsequent research endeavors have attempted to eliminate this assumption. Aria [30] designs optimistic deterministic concurrency control to execute transactions first and then determine the order. However, this method not only requires introducing 2PC-like communication but also inheriting the limitations of OCC, leading to a notable increase in aborts and a reduction in throughput during high contention workloads [25, 46]. Many other works, such as BCDB [32], Harmony [26] and DOCC [13] have adopted the same optimistic deterministic concurrency control concept. Similar to Aria, these approaches also inherit the drawbacks of OCC.

The second category is non-deterministic algorithms, which encompasses a wide range of algorithms, including 2PL [8, 9, 17, 19], OCC [18, 23, 24, 28, 31, 43, 50, 51], timestamp ordering [9], MVCC [16, 33–35, 47]. However, these algorithms inevitably require the 2PC protocol to address the serializability of distributed transactions, introducing significant network communication overhead. Furthermore, these algorithms also suffer from a substantial number of aborts in high contention scenarios [20, 21].

Hybrid Concurrency Control. Hybrid approaches aim to merge multiple algorithms into a singular one or incorporate multiple algorithms within a single database system. The examples of the former include MOCC [45], MVOCC [27], which integrate 2PL and MVCC with OCC, respectively. The latter mostly focuses on incorporating non-deterministic algorithms. HSync [37] and C3 [39] hybrid 2PL and OCC by dividing transactions into different categories. CormCC [40] mixes PartCC [22], 2PL, and OCC by dividing data partitions into different categories. Tebaldi [38] constructs hierarchical concurrency control by analyzing the stored procedures. Polyjuice [44] introduces reinforcement learning to design specific concurrency control for each stored procedure. Snapper [29] is the only work that mixed deterministic and non-deterministic (i.e., 2PL) algorithms so far. It only considers the proportion of transactions that pre-declare their read/write sets as the factor for concurrency control algorithm assignment, thereby failing to fully leverage the advantages of each algorithm. In addition, Snapper is a coarser-grained approach (conflict orders between 2PL transactions and Calvin batches) to processing transactions, introducing a high abort rate for 2PL. HDCC considers contention level and distribution transaction rate as the other two factors for assigning concurrency control algorithms, and designs a global validation to detect orders in a fine-grained manner.

8 CONCLUSION

In this paper, we present HDCC, a novel distributed hybrid approach that integrates Calvin and OCC. We propose lock-sharing and global validation mechanisms to guarantee the serializability of HDCC. Then we design a two-log-interleaving mechanism to ensure correct recovery upon failure. Besides, we propose a rule-based assignment mechanism, tailoring its optimal choice to different workload scenarios. Through comprehensive experimentation on YCSB and TPC-C benchmarks, HDCC is proven to excel over the state-of-the-art hybrid approach by providing up to 3.1× better throughput.

REFERENCES

[1] 2023. Cosmos DB. https://azure.microsoft.com/en-us/products/cosmos-db/.
[2] 2023. FaunaDB. https://fauna.com/.
[3] 2023. MonetDB. https://www.monetdb.org/.
[4] 2023. TPC-C. http://www.tpc.org/tpcc/.
[5] 2023. YugabyteDB. https://www.yugabyte.com/.
[6] Atul Adya, Barbara Liskov, and Patrick E. O'Neil. 2000. Generalized Isolation Level Definitions. In *Proceedings of the 16th International Conference on Data Engineering, San Diego, California, USA, February 28 - March 3, 2000*, David B. Lomet and Gerhard Weikum (Eds.). IEEE Computer Society, 67–78. https://doi.org/10.1109/ICDE.2000.839388
[7] Rakesh Agrawal, Michael J. Carey, and Miron Livny. 1987. Concurrency Control Performance Modeling: Alternatives and Implications. *ACM Trans. Database Syst.* 12, 4 (1987), 609–654. https://doi.org/10.1145/32204.32220
[8] Claude Barthels, Ingo Müller, Konstantin Taranov, Gustavo Alonso, and Torsten Hoefler. 2019. Strong consistency is not hard to get: Two-Phase Locking and Two-Phase Commit on Thousands of Cores. *Proc. VLDB Endow.* 12, 13 (2019), 2325–2338.
[9] Philip A. Bernstein and Nathan Goodman. 1981. Concurrency Control in Distributed Database Systems. *ACM Comput. Surv.* 13, 2 (1981), 185–221.
[10] Tuan Cao, Marcos Antonio Vaz Salles, Benjamin Sowell, Yao Yue, Alan J. Demers, Johannes Gehrke, and Walker M. White. 2011. Fast checkpoint recovery algorithms for frequently consistent applications. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2011, Athens, Greece, June 12-16, 2011*, Timos K. Sellis, Renée J. Miller, Anastasios Kementsietsidis, and Yannis Velegrakis (Eds.). ACM, 265–276. https://doi.org/10.1145/1989323.1989352
[11] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC 2010, Indianapolis, Indiana, USA, June 10-11, 2010*, Joseph M. Hellerstein, Surajit Chaudhuri, and Mendel Rosenblum (Eds.). ACM, 143–154. https://doi.org/10.1145/1807128.1807152
[12] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Åke Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. Hekaton: SQL server's memory-optimized OLTP engine. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*, Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias (Eds.). ACM, 1243–1254. https://doi.org/10.1145/2463676.2463710
[13] Zhiyuan Dong, Chuzhe Tang, Jia-Chen Wang, Zhaoguo Wang, Haibo Chen, and Binyu Zang. 2020. Optimistic Transaction Processing in Deterministic Database. *J. Comput. Sci. Technol.* 35, 2 (2020), 382–394. https://doi.org/10.1007/s11390-020-9700-5
[14] Jose M. Faleiro, Daniel Abadi, and Joseph M. Hellerstein. 2017. High Performance Transactions via Early Write Visibility. *Proc. VLDB Endow.* 10, 5 (2017), 613–624. https://doi.org/10.14778/3055540.3055553
[15] Jose M. Faleiro and Daniel J. Abadi. 2015. Rethinking serializable multiversion concurrency control. *Proc. VLDB Endow.* 8, 11 (2015), 1190–1201. https://doi.org/10.14778/2809974.2809981
[16] Jose M. Faleiro and Daniel J. Abadi. 2015. Rethinking serializable multiversion concurrency control. *Proc. VLDB Endow.* 8, 11 (2015), 1190–1201.
[17] Jim Gray, Raymond A. Lorie, Gianfranco R. Putzolu, and Irving L. Traiger. 1976. Granularity of Locks and Degrees of Consistency in a Shared Data Base. In *IFIP Working Conference on Modelling in Data Base Management Systems*. North-Holland, 365–394.
[18] Jinwei Guo, Peng Cai, Jiahao Wang, Weining Qian, and Aoying Zhou. 2019. Adaptive Optimistic Concurrency Control for Heterogeneous Workloads. *Proc. VLDB Endow.* 12, 5 (2019), 584–596.
[19] Zhihan Guo, Kan Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing Locks As Early As You Can: Reducing Contention of Hotspots by Violating Two-Phase Locking. In *SIGMOD Conference*. ACM, 658–670.
[20] Rachael Harding, Dana Van Aken, Andrew Pavlo, and Michael Stonebraker. 2017. An Evaluation of Distributed Concurrency Control. *Proc. VLDB Endow.* 10, 5 (2017), 553–564. https://doi.org/10.14778/3055540.3055548
[21] Yihe Huang, William Qian, Eddie Kohler, Barbara Liskov, and Liuba Shrira. 2020. Opportunities for Optimism in Contended Main-Memory Multicore Transactions. *Proc. VLDB Endow.* 13, 5 (2020), 629–642.
[22] Robert Kallman, Hideaki Kimura, Jonathan Natkins, Andrew Pavlo, Alex Rasin, Stanley B. Zdonik, Evan P. C. Jones, Samuel Madden, Michael Stonebraker, Yang Zhang, John Hugg, and Daniel J. Abadi. 2008. H-store: a high-performance, distributed main memory transaction processing system. *Proc. VLDB Endow.* 1, 2 (2008), 1496–1499. https://doi.org/10.14778/1454159.1454211
[23] Kangnyeon Kim, Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. 2016. ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads. In *SIGMOD Conference*. ACM, 1675–1687.
[24] H. T. Kung and John T. Robinson. 1981. On Optimistic Methods for Concurrency Control. *ACM Trans. Database Syst.* 6, 2 (1981), 213–226.

[25] Z. Lai, H. Fan, W. Zhou, Z. Ma, X. Peng, F. Li, and E. Lo. 2023. Knock Out 2PC with Practicality Intact: a High-performance and General Distributed Transaction Protocol. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, Los Alamitos, CA, USA, 2317–2331. https://doi.org/10.1109/ICDE55515.2023.00179

[26] Ziliang Lai, Chris Liu, and Eric Lo. 2023. When Private Blockchain Meets Deterministic Database. *Proc. ACM Manag. Data* 1, 1 (2023), 98:1–98:28. https://doi.org/10.1145/3588952

[27] Per-Åke Larson, Spyros Blanas, Cristian Diaconu, Craig Freedman, Jignesh M. Patel, and Mike Zwilling. 2011. High-Performance Concurrency Control Mechanisms for Main-Memory Databases. *Proc. VLDB Endow.* 5, 4 (2011), 298–309. https://doi.org/10.14778/2095686.2095689

[28] Hyeontaek Lim, Michael Kaminsky, and David G. Andersen. 2017. Cicada: Dependably Fast Multi-Core In-Memory Transactions. In *SIGMOD Conference*. ACM, 21–35.

[29] Yijian Liu, Li Su, Vivek Shah, Yongluan Zhou, and Marcos Antonio Vaz Salles. 2022. Hybrid Deterministic and Nondeterministic Execution of Transactions in Actor Systems. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 65–78. https://doi.org/10.1145/3514221.3526172

[30] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2020. Aria: A Fast and Practical Deterministic OLTP Database. *Proc. VLDB Endow.* 13, 11 (2020), 2047–2060. http://www.vldb.org/pvldb/vol13/p2047-lu.pdf

[31] Hatem A. Mahmoud, Vaibhav Arora, Faisal Nawab, Divyakant Agrawal, and Amr El Abbadi. 2014. MaaT: Effective and scalable coordination of distributed transactions in the cloud. *Proc. VLDB Endow.* 7, 5 (2014), 329–340. http://www.vldb.org/pvldb/vol7/p329-mahmoud.pdf

[32] Senthil Nathan, Chander Govindarajan, Adarsh Saraf, Manish Sethi, and Praveen Jayachandran. 2019. Blockchain Meets Database: Design and Implementation of a Blockchain Relational Database. *Proc. VLDB Endow.* 12, 11 (2019), 1539–1552. https://doi.org/10.14778/3342263.3342632

[33] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. 2015. Fast Serializable Multi-Version Concurrency Control for Main-Memory Database Systems. In *SIGMOD Conference*. ACM, 677–689.

[34] David P. Reed. 1978. *Naming and synchronization in a decentralized computer system*. Ph.D. Dissertation. Massachusetts Institute of Technology, Cambridge, MA, USA. https://hdl.handle.net/1721.1/16279

[35] David P. Reed. 1983. Implementing Atomic Actions on Decentralized Data. *ACM Trans. Comput. Syst.* 1, 1 (1983), 3–23. https://doi.org/10.1145/357353.357355

[36] Kun Ren, Alexander Thomson, and Daniel J. Abadi. 2014. An Evaluation of the Advantages and Disadvantages of Deterministic Database Systems. *Proc. VLDB Endow.* 7, 10 (jun 2014), 821–832. https://doi.org/10.14778/2732951.2732955

[37] Zechao Shang, Feifei Li, Jeffrey Xu Yu, Zhiwei Zhang, and Hong Cheng. 2016. Graph Analytics Through Fine-Grained Parallelism. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Fatma Özcan, Georgia Koutrika, and Sam Madden (Eds.). ACM, 463–478. https://doi.org/10.1145/2882903.2915238

[38] Chunzhi Su, Natacha Crooks, Cong Ding, Lorenzo Alvisi, and Chao Xie. 2017. Bringing Modular Concurrency Control to the Next Level. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*, Semih Salihoglu, Wenchao Zhou, Rada Chirkova, Jun Yang, and Dan Suciu (Eds.). ACM, 283–297. https://doi.org/10.1145/3035918.3064031

[39] Xuebin Su, Hongzhi Wang, and Yan Zhang. 2021. Concurrency Control Based on Transaction Clustering. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 2195–2200. https://doi.org/10.1109/ICDE51399.2021.00223

[40] Dixin Tang and Aaron J. Elmore. 2018. Toward Coordination-free and Reconfigurable Mixed Concurrency Control. In *2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018*, Haryadi S. Gunawi and Benjamin Reed (Eds.). USENIX Association, 809–822. https://www.usenix.org/conference/atc18/presentation/tang

[41] Dixin Tang, Hao Jiang, and Aaron J. Elmore. 2017. Adaptive Concurrency Control: Despite the Looking Glass, One Concurrency Control Does Not Fit All. In *8th Biennial Conference on Innovative Data Systems Research, CIDR 2017, Chaminade, CA, USA, January 8-11, 2017, Online Proceedings*. www.cidrdb.org. http://cidrdb.org/cidr2017/papers/p63-tang-cidr17.pdf

[42] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, Scottsdale, AZ, USA, May 20-24, 2012*, K. Selçuk Candan, Yi Chen, Richard T. Snodgrass, Luis Gravano, and Ariel Fuxman (Eds.). ACM, 1–12. https://doi.org/10.1145/2213836.2213838

[43] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *ACM SIGOPS 24th Symposium on Operating Systems Principles, SOSP '13, Farmington, PA, USA, November 3-6, 2013*, Michael Kaminsky and Mike Dahlin (Eds.). ACM, 18–32.

[44] Jia-Chen Wang, Ding Ding, Huan Wang, Conrad Christensen, Zhaoguo Wang, Haibo Chen, and Jinyang Li. 2021. Polyjuice: High-Performance Transactions via Learned Concurrency Control. In *15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2021, July 14-16, 2021*, Angela Demke Brown and Jay R. Lorch (Eds.). USENIX Association, 198–216. https://www.usenix.org/conference/osdi21/presentation/wang-jiachen

[45] Tianzheng Wang and Hideaki Kimura. 2016. Mostly-Optimistic Concurrency Control for Highly Contended Dynamic Workloads on a Thousand Cores. *Proc. VLDB Endow.* 10, 2 (2016), 49–60. https://doi.org/10.14778/3015274.3015276

[46] Yang Wang, Miao Yu, Yujie Hui, Fang Zhou, Yuyang Huang, Rui Zhu, Xueyuan Ren, Tianxi Li, and Xiaoyi Lu. 2022. A Study of Database Performance Sensitivity to Experiment Settings. *Proc. VLDB Endow.* 15, 7 (mar 2022), 1439–1452. https://doi.org/10.14778/3523210.3523221

[47] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. *Proc. VLDB Endow.* 10, 7 (2017), 781–792. https://doi.org/10.14778/3067421.3067427

[48] Chao Xie, Chunzhi Su, Cody Littley, Lorenzo Alvisi, Manos Kapritsos, and Yang Wang. 2015. High-performance ACID via modular concurrency control. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, Ethan L. Miller and Steven Hand (Eds.). ACM, 279–294. https://doi.org/10.1145/2815400.2815430

[49] Xiangyao Yu, George Bezerra, Andrew Pavlo, Srinivas Devadas, and Michael Stonebraker. 2014. Staring into the Abyss: An Evaluation of Concurrency Control with One Thousand Cores. *Proc. VLDB Endow.* 8, 3 (2014), 209–220. https://doi.org/10.14778/2735508.2735511

[50] Xiangyao Yu, Andrew Pavlo, Daniel Sánchez, and Srinivas Devadas. 2016. TicToc: Time Traveling Optimistic Concurrency Control. In *SIGMOD Conference*. ACM, 1629–1642.

[51] Xiangyao Yu, Yu Xia, Andrew Pavlo, Daniel Sánchez, Larry Rudolph, and Srinivas Devadas. 2018. Sundial: Harmonizing Concurrency Control and Caching in a Distributed OLTP Database Management System. *Proc. VLDB Endow.* 11, 10 (2018), 1289–1302.

Appendix

MODULE *HDCC*

EXTENDS *Integers*, *Sequences*, *TLC*

CONSTANTS
N,
M,
FailN

--algorithm *hdcc*
variables
Nodes $= 1 \,..\, N$;
Transactions $= 1 \,..\, 2 * M$;
CalvinTransactions $= 1 \,..\, M$;
OCCTransactions $= M + 1 \,..\, 2 * M$;
FailNode $= 1 \,..\, FailN$;

database $= [n \in Nodes \mapsto [value \mapsto 0,\ cid \mapsto 0,\ cid\_ \mapsto 0]]$;

calvinLog $= [n \in Nodes \mapsto \langle\rangle]$;

occLog $= [n \in Nodes \mapsto \langle\rangle]$;

gMaxCid $= [n \in OCCTransactions \mapsto 0]$;

systemState = "Normal" ;

transactions $= [n \in Transactions \mapsto$ FALSE$]$;

define
GetMaxCid(*dataItem*) $\triangleq$
LET *maxCid* $\triangleq$ IF *dataItem.cid* $>$ *dataItem.cid_* THEN *dataItem.cid* ELSE *dataItem.cid_*IN
maxCid
end define ;

macro *WriteCalvinLog*(*txnId*, *dataItem*, *newValue*)**begin**
calvinLog[*dataItem*] := *Append*(*calvinLog*[*dataItem*],
$[txnId \mapsto txnId,\ dataItem \mapsto dataItem,\ newValue \mapsto newValue]$) ;
end macro ;

macro *WriteOCCLog*(*txnId*, *dataItem*, *newValue*, *gMaxCid*)**begin**
occLog[*dataItem*] := *Append*(*occLog*[*dataItem*],
$[txnId \mapsto txnId,\ dataItem \mapsto dataItem,\ newValue \mapsto newValue,\ gMaxCid \mapsto gMaxCid]$) ;
end macro ;

fair process *calvinTxn* $\in$ *CalvinTransactions*
variables
dataItem $\in$ *Nodes*, *newValue* = *self* ;
begin
CalvinTxnProcess:
WriteCalvinLog(*newValue*, *dataItem*, *newValue*) ;

database[*dataItem*] := $[value \mapsto newValue,\ cid \mapsto newValue,\ cid\_ \mapsto database[dataItem].cid\_]$;
transactions[*self*] := TRUE ;
end process ;

fair process *occTxn* $\in$ *OCCTransactions*
variables
dataItem $\in$ *Nodes* ;
newValue = *self* ;
localMaxCid $= 0$;
begin *occTxnProcess*:
database[*dataItem*].*value* := *newValue* ;

```
  localMaxCid := GetMaxCid(database[dataItem]) ;
  gMaxCid[self] := IF gMaxCid[self] > localMaxCid THEN gMaxCid[self] ELSE localMaxCid ;

  if gMaxCid[self] > database[dataItem].cid_ then
     LargerCid:
    database[dataItem] := [value ↦ newValue, cid ↦ database[dataItem].cid, cid_ ↦ gMaxCid[self]] ;
   else
     SmallerCid:
    database[dataItem].value := newValue ;
  end if ;

  WriteOCCLog:
    WriteOCCLog(newValue, dataItem, newValue, gMaxCid[self]) ;
    transactions[self] := TRUE ;
end process ;

process nodeFailure ∈ FailNode
variables
  node = self ;
  occLogIndex = 1 ;
  calvinLogIndex    = 1 ;
  currentGMaxCid = 0 ;
  occLogRecord = [txnId ↦ 0, dataItem ↦ 0, newValue ↦ 0] ;
  calvinLogRecord = [txnId ↦ 0, dataItem ↦ 0, newValue ↦ 0, gMaxCid ↦ 0] ;
begin
  Failure:
    await ∀ txn ∈ Transactions : txn = TRUE ;

    systemState := "Failure" ;
    database[self] := [value ↦ 0, cid ↦ 0, cid_ ↦ 0] ;

  Recovery:
    while Len(occLog[node]) > 0 do
        occLogRecord := Head(occLog[node]) ;
        occLog[node]  := Tail(occLog[node])  ;
        currentGMaxCid := occLogRecord.gMaxCid ;

            ApplyCalvinLog:
            while Len(calvinLog[node]) > 0 ∧ Head(calvinLog[node]).txnId ≤ currentGMaxCid do
                calvinLogRecord := Head(calvinLog[node]) ;
                calvinLog[node]  := Tail(calvinLog[node])  ;
                database[calvinLogRecord.dataItem] := [value ↦ calvinLogRecord.newValue,
                                                       cid ↦ calvinLogRecord.newValue,
                                                       cid_ ↦ database[calvinLogRecord.dataItem].cid_] ;
            end while ;

        database[occLogRecord.dataItem] := [value ↦ occLogRecord.newValue,
                                            cid ↦ database[occLogRecord.dataItem].cid,
                                            cid_ ↦ occLogRecord.gMaxCid] ;
    end while ;

    ApplyRemaining:
    while Len(calvinLog[node]) > 0 do
        calvinLogRecord := Head(calvinLog[node]) ;
        calvinLog[node]  := Tail(calvinLog[node])  ;
        database[calvinLogRecord.dataItem] := [value ↦ calvinLogRecord.newValue,
                                               cid ↦ calvinLogRecord.newValue,
                                               cid_ ↦ database[calvinLogRecord.dataItem].cid_] ;
    end while ;
end process ;
end algorithm  ;
 BEGIN TRANSLATION (chksum(pcal) = "479d81c1" ∧ chksum(tla) = "370b93cf")
```

Process variable *dataItem* of process *calvinTxn* at line 51 col 3 changed to *dataItem_*

Process variable *newValue* of process *calvinTxn* at line 51 col 23 changed to *newValue_*

VARIABLES *Nodes*, *Transactions*, *CalvinTransactions*, *OCCTransactions*, *FailNode*, *database*, *calvinLog*, *occLog*, *gMaxCid*, *systemState*, *transactions*, *pc*

define statement

$GetMaxCid(dataItem) \triangleq$
LET $maxCid \triangleq$ IF $dataItem.cid > dataItem.cid\_$ THEN $dataItem.cid$ ELSE $dataItem.cid\_$ IN
$maxCid$

VARIABLES *dataItem_*, *newValue_*, *dataItem*, *newValue*, *localMaxCid*, *node*, *occLogIndex*, *calvinLogIndex*, *currentGMaxCid*, *occLogRecord*, *calvinLogRecord*

$vars \triangleq \langle Nodes, Transactions, CalvinTransactions, OCCTransactions, FailNode, database, calvinLog, occLog, gMaxCid, systemState, transactions, pc, dataItem\_, newValue\_, dataItem, newValue, localMaxCid, node, occLogIndex, calvinLogIndex, currentGMaxCid, occLogRecord, calvinLogRecord\rangle$

$ProcSet \triangleq (CalvinTransactions) \cup (OCCTransactions) \cup (FailNode)$

$Init \triangleq$ Global variables
$\land Nodes = 1 \,..\, N$
$\land Transactions = 1 \,..\, 2 * M$
$\land CalvinTransactions = 1 \,..\, M$
$\land OCCTransactions = M + 1 \,..\, 2 * M$
$\land FailNode = 1 \,..\, FailN$
$\land database = [n \in Nodes \mapsto [value \mapsto 0,\ cid \mapsto 0,\ cid\_ \mapsto 0]]$
$\land calvinLog = [n \in Nodes \mapsto \langle\rangle]$
$\land occLog = [n \in Nodes \mapsto \langle\rangle]$
$\land gMaxCid = [n \in OCCTransactions \mapsto 0]$
$\land systemState =$ "Normal"
$\land transactions = [n \in Transactions \mapsto$ FALSE$]$
Process *calvinTxn*
$\land dataItem\_ \in [CalvinTransactions \to Nodes]$
$\land newValue\_ = [self \in CalvinTransactions \mapsto self]$
Process *occTxn*
$\land dataItem \in [OCCTransactions \to Nodes]$
$\land newValue = [self \in OCCTransactions \mapsto self]$
$\land localMaxCid = [self \in OCCTransactions \mapsto 0]$
Process *nodeFailure*
$\land node = [self \in FailNode \mapsto self]$
$\land occLogIndex = [self \in FailNode \mapsto 1]$
$\land calvinLogIndex = [self \in FailNode \mapsto 1]$
$\land currentGMaxCid = [self \in FailNode \mapsto 0]$
$\land occLogRecord = [self \in FailNode \mapsto [txnId \mapsto 0,\ dataItem \mapsto 0,\ newValue \mapsto 0]]$
$\land calvinLogRecord = [self \in FailNode \mapsto [txnId \mapsto 0,\ dataItem \mapsto 0,\ newValue \mapsto 0,\ gMaxCid \mapsto 0]]$
$\land pc = [self \in ProcSet \mapsto$ CASE $self \in CalvinTransactions \to$ "CalvinTxnProcess"
$\square\ self \in OCCTransactions \to$ "occTxnProcess"
$\square\ self \in FailNode \to$ "Failure"$]$

$CalvinTxnProcess(self) \triangleq$
$\land pc[self] =$ "CalvinTxnProcess"
$\land calvinLog' = [calvinLog$ EXCEPT $![dataItem\_[self]] = Append(calvinLog[dataItem\_[self]],$
$[txnId \mapsto newValue\_[self],\ dataItem \mapsto dataItem\_[self],\ newValue \mapsto newValue\_[self]])]$
$\land database' = [database$ EXCEPT $![dataItem\_[self]] =$
$[value \mapsto newValue\_[self],\ cid \mapsto newValue\_[self],\ cid\_ \mapsto database[dataItem\_[self]].cid\_]]$
$\land transactions' = [transactions$ EXCEPT $![self] =$ TRUE$]$
$\land pc' = [pc$ EXCEPT $![self] =$ "Done"$]$
$\land$ UNCHANGED $\langle Nodes, Transactions,$
$CalvinTransactions, OCCTransactions,$

$FailNode$, $occLog$, $gMaxCid$,
$systemState$, $dataItem\_$, $newValue\_$,
$dataItem$, $newValue$, $localMaxCid$,
$node$, $occLogIndex$, $calvinLogIndex$,
$currentGMaxCid$, $occLogRecord$,
$calvinLogRecord\rangle$

$calvinTxn(self) \triangleq CalvinTxnProcess(self)$

$occTxnProcess(self) \triangleq$
$\land\ pc[self] =$ "occTxnProcess"
$\land\ database' = [database$ EXCEPT $![dataItem[self]].value = newValue[self]]$
$\land\ localMaxCid' = [localMaxCid$ EXCEPT $![self] = GetMaxCid(database'[dataItem[self]])]$
$\land\ gMaxCid' = [gMaxCid$ EXCEPT $![self] =$ IF $gMaxCid[self] > localMaxCid'[self]$
THEN $gMaxCid[self]$
ELSE $localMaxCid'[self]]$
$\land$ IF $gMaxCid'[self] > database'[dataItem[self]].cid\_$
THEN $\land\ pc' = [pc$ EXCEPT $![self] =$ "LargerCid"$]$
ELSE $\land\ pc' = [pc$ EXCEPT $![self] =$ "SmallerCid"$]$
$\land$ UNCHANGED $\langle Nodes, Transactions, CalvinTransactions,$
$OCCTransactions, FailNode, calvinLog,$
$occLog, systemState, transactions,$
$dataItem\_, newValue\_, dataItem,$
$newValue, node, occLogIndex,$
$calvinLogIndex, currentGMaxCid,$
$occLogRecord, calvinLogRecord\rangle$

$LargerCid(self) \triangleq$
$\land\ pc[self] =$ "LargerCid"
$\land\ database' = [database$ EXCEPT $![dataItem[self]] = [value \mapsto newValue[self],$
$cid \mapsto database[dataItem[self]].cid,\ cid\_ \mapsto gMaxCid[self]]]$
$\land\ pc' = [pc$ EXCEPT $![self] =$ "WriteOCCLog"$]$
$\land$ UNCHANGED $\langle Nodes, Transactions, CalvinTransactions,$
$OCCTransactions, FailNode, calvinLog,$
$occLog, gMaxCid, systemState, transactions,$
$dataItem\_, newValue\_, dataItem, newValue,$
$localMaxCid, node, occLogIndex,$
$calvinLogIndex, currentGMaxCid,$
$occLogRecord, calvinLogRecord\rangle$

$SmallerCid(self) \triangleq$
$\land\ pc[self] =$ "SmallerCid"
$\land\ database' = [database$ EXCEPT $![dataItem[self]].value = newValue[self]]$
$\land\ pc' = [pc$ EXCEPT $![self] =$ "WriteOCCLog"$]$
$\land$ UNCHANGED $\langle Nodes, Transactions, CalvinTransactions,$
$OCCTransactions, FailNode, calvinLog,$
$occLog, gMaxCid, systemState, transactions,$
$dataItem\_, newValue\_, dataItem, newValue,$
$localMaxCid, node, occLogIndex,$
$calvinLogIndex, currentGMaxCid,$
$occLogRecord, calvinLogRecord\rangle$

$WriteOCCLog(self) \triangleq$
$\land\ pc[self] =$ "WriteOCCLog"
$\land\ occLog' = [occLog$ EXCEPT $![dataItem[self]] = Append(occLog[dataItem[self]],$
$[txnId \mapsto newValue[self],\ dataItem \mapsto dataItem[self],$
$newValue \mapsto newValue[self],\ gMaxCid \mapsto (gMaxCid[self])])]$
$\land\ transactions' = [transactions$ EXCEPT $![self] =$ TRUE$]$
$\land\ pc' = [pc$ EXCEPT $![self] =$ "Done"$]$
$\land$ UNCHANGED $\langle Nodes, Transactions, CalvinTransactions,$
$OCCTransactions, FailNode, database,$
$calvinLog, gMaxCid, systemState,$
$dataItem\_, newValue\_, dataItem, newValue,$
$localMaxCid, node, occLogIndex,$
$calvinLogIndex, currentGMaxCid,$

$occLogRecord,\ calvinLogRecord\rangle$

$occTxn(self) \triangleq occTxnProcess(self) \lor LargerCid(self) \lor SmallerCid(self)$
$\quad \lor WriteOCCLog(self)$

$Failure(self) \triangleq \land pc[self] = \text{"Failure"}$
$\land \forall\, txn \in Transactions : txn = \text{TRUE}$
$\land systemState' = \text{"Failure"}$
$\land database' = [database \text{ EXCEPT } ![self] = [value \mapsto 0,\ cid \mapsto 0,\ cid\_ \mapsto 0]]$
$\land pc' = [pc \text{ EXCEPT } ![self] = \text{"Recovery"}]$
$\land \text{UNCHANGED } \langle Nodes,\ Transactions,\ CalvinTransactions,$
$OCCTransactions,\ FailNode,\ calvinLog,\ occLog,$
$gMaxCid,\ transactions,\ dataItem\_,\ newValue\_,$
$dataItem,\ newValue,\ localMaxCid,\ node,$
$occLogIndex,\ calvinLogIndex,\ currentGMaxCid,$
$occLogRecord,\ calvinLogRecord\rangle$

$Recovery(self) \triangleq \land pc[self] = \text{"Recovery"}$
$\land \text{IF } Len(occLog[node[self]]) > 0$
$\text{THEN } \land occLogRecord' = [occLogRecord \text{ EXCEPT } ![self] = Head(occLog[node[self]])]$
$\land occLog' = [occLog \text{ EXCEPT } ![node[self]] = Tail(occLog[node[self]])]$
$\land currentGMaxCid' = [currentGMaxCid \text{ EXCEPT } ![self] = occLogRecord'[self].gMaxCid]$
$\land pc' = [pc \text{ EXCEPT } ![self] = \text{"ApplyCalvinLog"}]$
$\text{ELSE } \land pc' = [pc \text{ EXCEPT } ![self] = \text{"ApplyRemaining"}]$
$\land \text{UNCHANGED } \langle occLog,\ currentGMaxCid,$
$occLogRecord\rangle$
$\land \text{UNCHANGED } \langle Nodes,\ Transactions,\ CalvinTransactions,$
$OCCTransactions,\ FailNode,\ database,$
$calvinLog,\ gMaxCid,\ systemState,$
$transactions,\ dataItem\_,\ newValue\_,\ dataItem,$
$newValue,\ localMaxCid,\ node,\ occLogIndex,$
$calvinLogIndex,\ calvinLogRecord\rangle$

$ApplyCalvinLog(self) \triangleq \land pc[self] = \text{"ApplyCalvinLog"}$
$\land \text{IF } Len(calvinLog[node[self]]) > 0 \land Head(calvinLog[node[self]]).txnId \leq currentGMaxCid[self]$
$\text{THEN } \land calvinLogRecord' = [calvinLogRecord \text{ EXCEPT } ![self] = Head(calvinLog[node[self]])]$
$\land calvinLog' = [calvinLog \text{ EXCEPT } ![node[self]] = Tail(calvinLog[node[self]])]$
$\land database' = [database \text{ EXCEPT } ![calvinLogRecord'[self].dataItem] =$
$[value \mapsto calvinLogRecord'[self].newValue,$
$cid \mapsto calvinLogRecord'[self].newValue,$
$cid\_ \mapsto database[calvinLogRecord'[self].dataItem].cid\_]]$
$\land pc' = [pc \text{ EXCEPT } ![self] = \text{"ApplyCalvinLog"}]$
$\text{ELSE } \land database' = [database \text{ EXCEPT } ![occLogRecord[self].dataItem] =$
$[value \mapsto occLogRecord[self].newValue,$
$cid \mapsto database[occLogRecord[self].dataItem].cid,$
$cid\_ \mapsto occLogRecord[self].gMaxCid]]$
$\land pc' = [pc \text{ EXCEPT } ![self] = \text{"Recovery"}]$
$\land \text{UNCHANGED } \langle calvinLog,\ calvinLogRecord\rangle$
$\land \text{UNCHANGED } \langle Nodes,\ Transactions,$
$CalvinTransactions,\ OCCTransactions,$
$FailNode,\ occLog,\ gMaxCid,\ systemState,$
$transactions,\ dataItem\_,\ newValue\_,$
$dataItem,\ newValue,\ localMaxCid,\ node,$
$occLogIndex,\ calvinLogIndex,$
$currentGMaxCid,\ occLogRecord\rangle$

$ApplyRemaining(self) \triangleq \land pc[self] = \text{"ApplyRemaining"}$
$\land \text{IF } Len(calvinLog[node[self]]) > 0$
$\text{THEN } \land calvinLogRecord' = [calvinLogRecord \text{ EXCEPT } ![self] = Head(calvinLog[node[self]])]$
$\land calvinLog' = [calvinLog \text{ EXCEPT } ![node[self]] = Tail(calvinLog[node[self]])]$
$\land database' = [database \text{ EXCEPT } ![calvinLogRecord'[self].dataItem] =$

$$
\begin{array}{l}
\qquad\qquad\qquad\qquad\qquad\qquad [value \mapsto calvinLogRecord'[self].newValue, \\
\qquad\qquad\qquad\qquad\qquad\qquad\ cid \mapsto calvinLogRecord'[self].newValue, \\
\qquad\qquad\qquad\qquad\qquad\qquad\ cid\_ \mapsto database[calvinLogRecord'[self].dataItem].cid\_] \\
\qquad\qquad\qquad\qquad \land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“ApplyRemaining”}] \\
\qquad\qquad \text{ELSE} \quad \land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“Done”}] \\
\qquad\qquad\qquad\qquad \land \text{ UNCHANGED } \langle database,\ calvinLog, \\
\qquad\qquad\qquad\qquad\qquad\qquad\ calvinLogRecord \rangle \\
\qquad\qquad \land \text{ UNCHANGED } \langle Nodes,\ Transactions, \\
\qquad\qquad\qquad\qquad\qquad\ CalvinTransactions,\ OCCTransactions, \\
\qquad\qquad\qquad\qquad\qquad\ FailNode,\ occLog,\ gMaxCid,\ systemState, \\
\qquad\qquad\qquad\qquad\qquad\ transactions,\ dataItem\_,\ newValue\_, \\
\qquad\qquad\qquad\qquad\qquad\ dataItem,\ newValue,\ localMaxCid,\ node, \\
\qquad\qquad\qquad\qquad\qquad\ occLogIndex,\ calvinLogIndex, \\
\qquad\qquad\qquad\qquad\qquad\ currentGMaxCid,\ occLogRecord \rangle
\end{array}
$$

$$
\begin{array}{l}
nodeFailure(self) \triangleq\ Failure(self) \lor Recovery(self) \\
\qquad\qquad\qquad\qquad \lor ApplyCalvinLog(self) \lor ApplyRemaining(self)
\end{array}
$$

Allow infinite stuttering to prevent deadlock on termination.

$$
\begin{array}{l}
Terminating \triangleq\ \land \forall\, self \in ProcSet : pc[self] = \text{“Done”} \\
\qquad\qquad\qquad\ \land \text{ UNCHANGED } vars
\end{array}
$$

$$
\begin{array}{l}
Next \triangleq\ (\exists\, self \in CalvinTransactions : calvinTxn(self)) \\
\qquad\quad \lor (\exists\, self \in OCCTransactions : occTxn(self)) \\
\qquad\quad \lor (\exists\, self \in FailNode : nodeFailure(self)) \\
\qquad\quad \lor Terminating
\end{array}
$$

$$
\begin{array}{l}
Spec \triangleq\ \land Init \land \Box[Next]_{vars} \\
\qquad\quad \land \forall\, self \in CalvinTransactions : \mathrm{WF}_{vars}(calvinTxn(self)) \\
\qquad\quad \land \forall\, self \in OCCTransactions : \mathrm{WF}_{vars}(occTxn(self))
\end{array}
$$

$$
Termination \triangleq\ \Diamond(\forall\, self \in ProcSet : pc[self] = \text{“Done”})
$$

END TRANSLATION